

A RESEARCH PRESENTATION

As Rocks May Think

Principle of AlphaGo | Automating Research with LLM tools

ZHOUYU JIN

MEDICAL SCHOOL | NANJING UNIVERSITY

August 16, 2026

RESOURCES & REFERENCES



What rebuilding AlphaGo teaches us about self-play, RL, and future of LLMs
- Eric Jang

As Rocks May Think

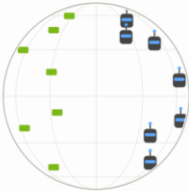
Feb 4, 2026

Whenever logical processes of thought are employed – that is, whenever thought for a time runs along an acceptive groove – there is an opportunity for the machine.

— Dr. Vannevar Bush, *As We May Think*, 1945

If we consider life to be a sort of open-ended MMO, the game server has just received a major update. All players take note: consider playing differently.

The world has changed a lot since 2022. ChatGPT happened. You can now ask it to [construct novel proofs of Erdos problems](#). Nation states are using [AI to automate cyberattacks](#). You can [pre-order a general purpose home humanoid](#). The Chinese robotics ecosystem is creating more open robots, data, and research than everyone else. Most big tech companies all have a humanoid project in the works. AI generated videos are indistinguishable from reality. The entire global economy is [re-organizing](#) around the [scale-up of AI models](#).



Chief among all changes is that machines can code and think quite well now.

Like many others, I spent the last 2 months on a [Claude Code bender](#), grappling with the fact that I no longer need to write code by hand anymore. I've been implementing AlphaGo from scratch (repo will be open sourced soon) to catch up on foundational deep learning techniques, and also to re-learn how to program with the full power of modern coding agents. I've set up Claude to not only write my infra and research ideas, but also propose hypotheses, draw conclusions, and suggest what experiments to try next. For those of you reading on desktop & tablet, the right side of this page shows examples of real prompts that I asked Claude to write for me.

For my "automated AlphaGo researcher" codebase, I created a Claude command [/experiment](#) which standardizes an "action" in the AlphaGo research environment as follows:

1. Create a self-contained experiment folder with datetime prefix and descriptive slug.
2. Write an experiment routine to a single-file python file and execute it.
3. Intermediate artifacts and data are saved to data/ and figures/ subdirectories. All files are stored in easy-to-parse formats like CSV

```
> /experiment Apply maximal update parameterization to find the best hyperparameters as I scale up. Start with GoResNet-100M as base. Use d-muP for depth-wise stability. Train 1 epoch on dev-train-100k. Submit up to 4 parallel Ray jobs.
```

As Rocks May Think | Eric Jang

AS ROCKS
MAY THINK

COVER

STORY

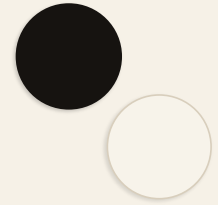
METHOD

EVIDENCE

FINDINGS

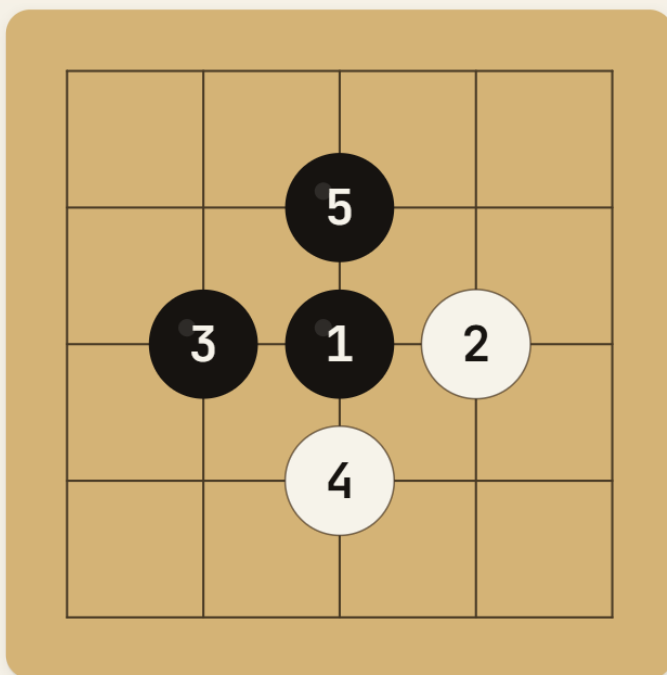
PART 01 • PROBLEM SETTING

01 • INTRO TO GO



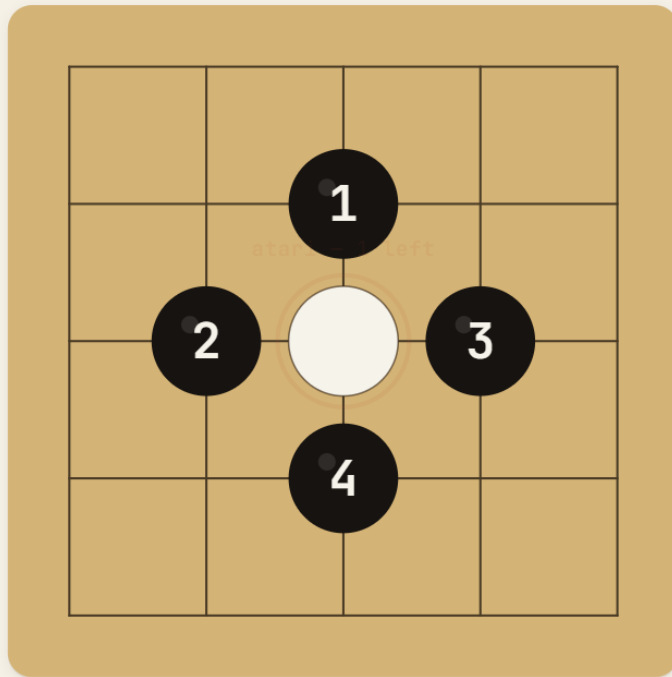
01 Placing Stones

Black plays first. Players alternate, placing one stone per turn on any empty intersection — not inside the squares.



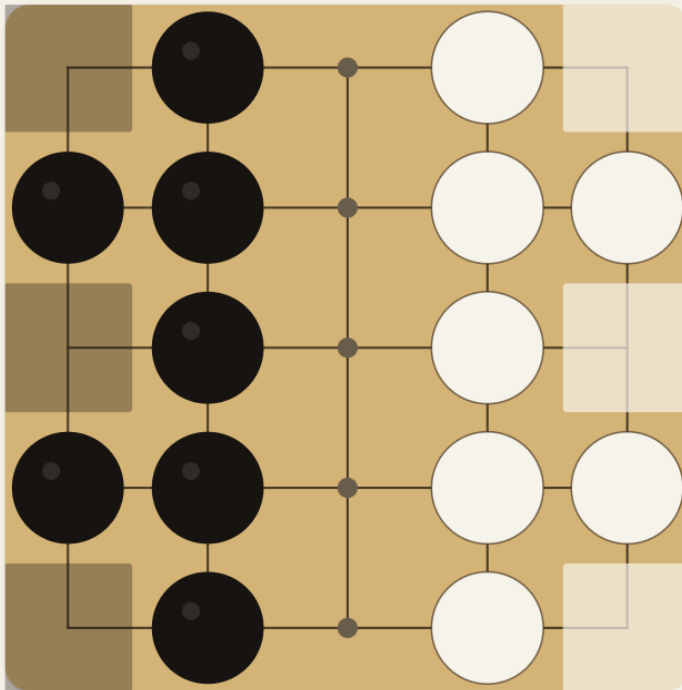
02 Capture

A stone's liberties are the empty points directly next to it. Fill every last liberty of an enemy group and it comes off the board.



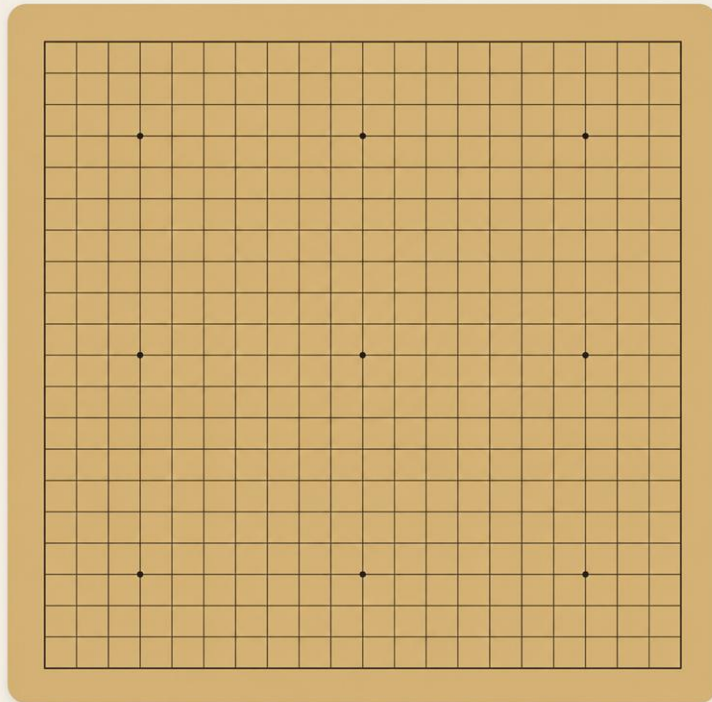
03 Ending the Game

When neither player can gain territory or captures, they *pass*.
Two consecutive passes end the game. Then the board is counted.



04 Why so hard to build a superhuman Go software?

19 × 19 board, about 361^{300} possible move sequences—far more than the number of atoms in the universe.



AS ROCKS
MAY THINK

COVER

STORY

METHOD

EVIDENCE

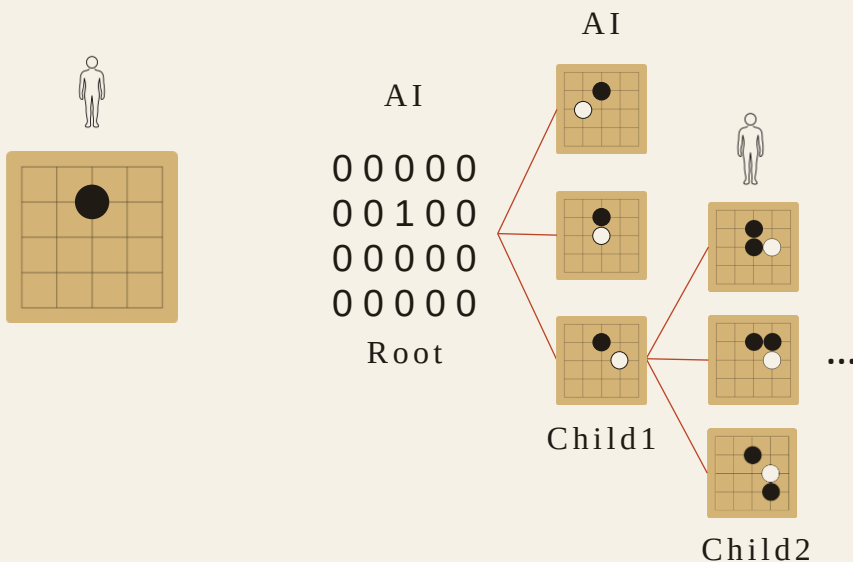
FINDINGS

PART 02 • REINFORCEMENT LERANING BASED ON MCTS

02 · PRINCIPLE OF ALPHAGO



01 Monte Carlo Tree Search (MCTS)



Four steps of MCTS

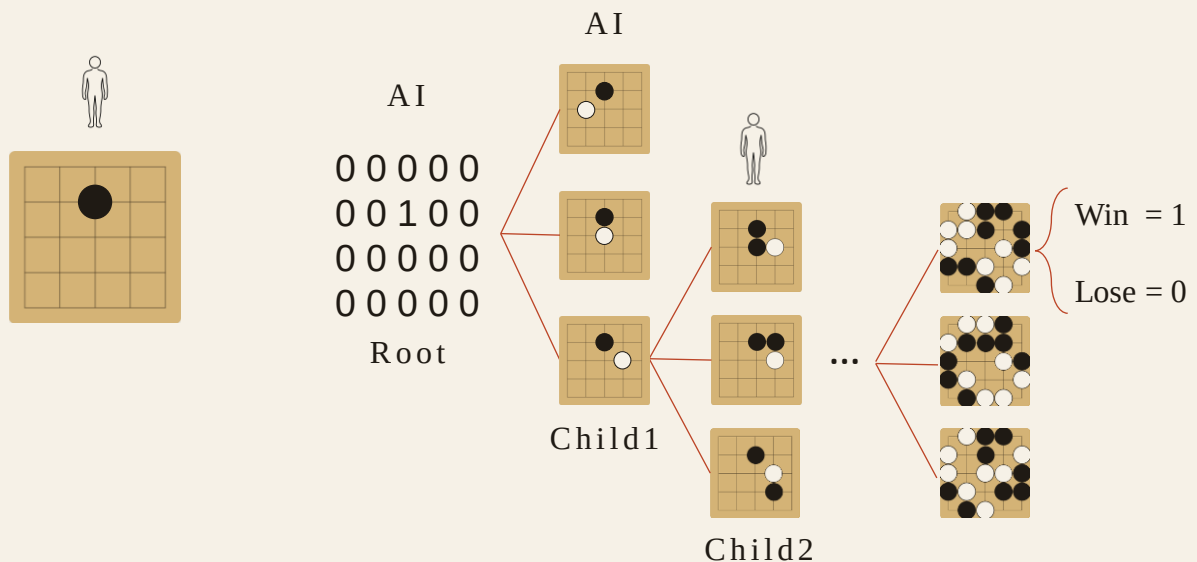
- Selection
- Expansion
- Evaluation
- Backup

Predicted Upper Confidence with Trees (PUCT) Algorithm

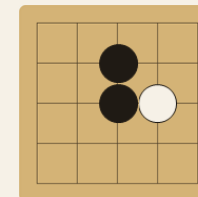
$$a_t = \operatorname{argmax}_a \left(Q(s, a) + c_{PUCT} P(s, a) \frac{\sqrt{N_{root}}}{1 + N(s, a)} \right)$$

- N_{root} : Over all visit count of root node
- $N(s, a)$: Visit count of action a
- $Q(s, a)$: Mean value of action a
- $P(s, a)$: Probability of choosing action a

01 Monte Carlo Tree Search (MCTS)

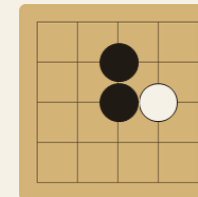


Human intuitions:
 $V_{\theta}(s) \rightarrow P(\text{win}) = ?$



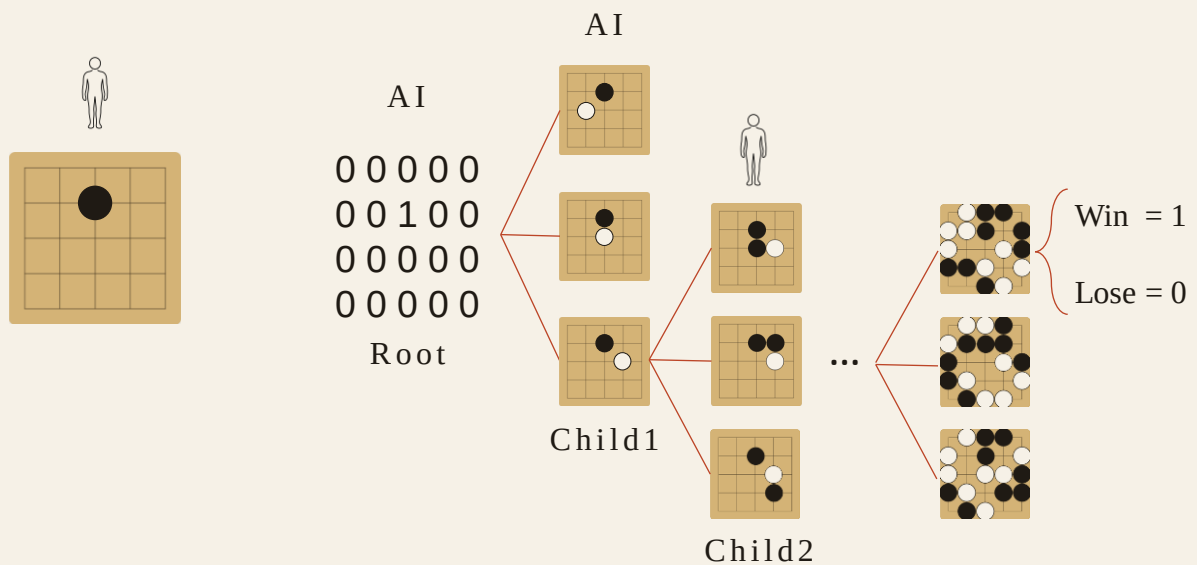
Can I win?

$p_{\theta}(a|s) \rightarrow P(a)$

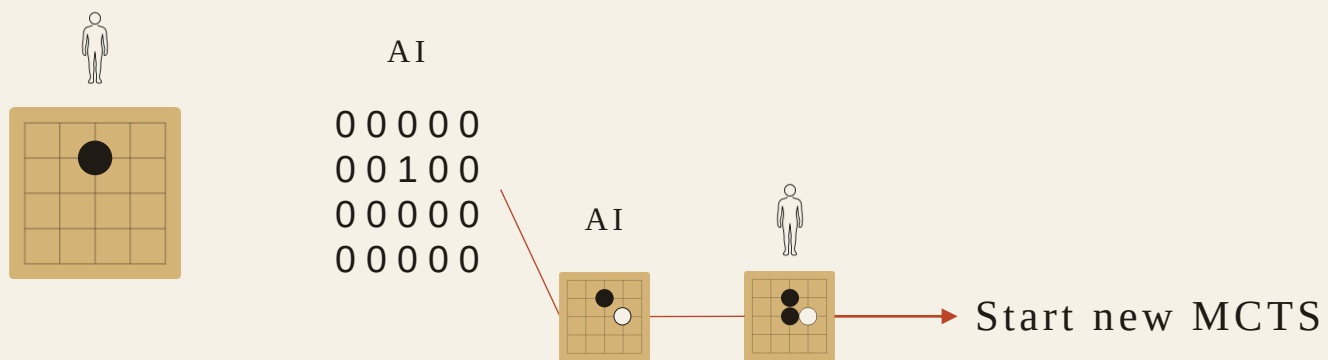


What are possible candidate moves?

01 Monte Carlo Tree Search (MCTS)



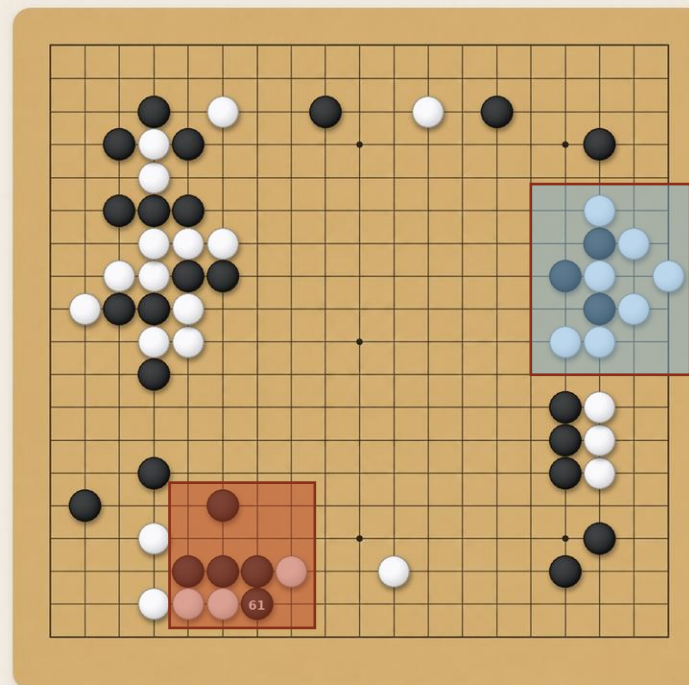
01 Monte Carlo Tree Search (MCTS)



02 Policy Network and Value Network

$$V_{\theta}(s) \rightarrow P(\text{win}) = ?$$

$$p_{\theta}(a|s) \rightarrow$$

02 Policy Network and Value Network

Go is a *perfect information game*.

If you know your opponent's game strategy π_{opp} , you can train your strategy based on π_{opp} :

$$\pi(\pi_{opp}) = \operatorname{argmax}_{\pi} V(\pi, \pi_{opp})$$

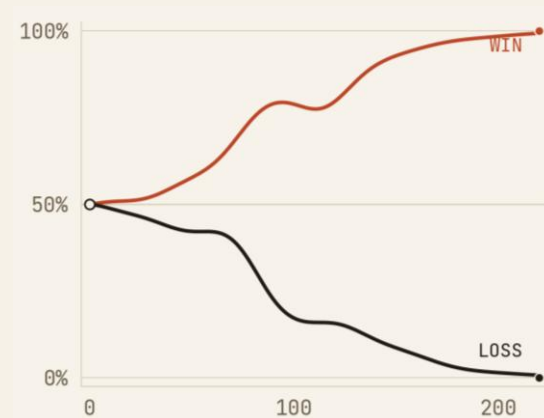
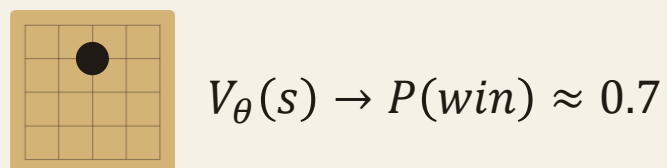
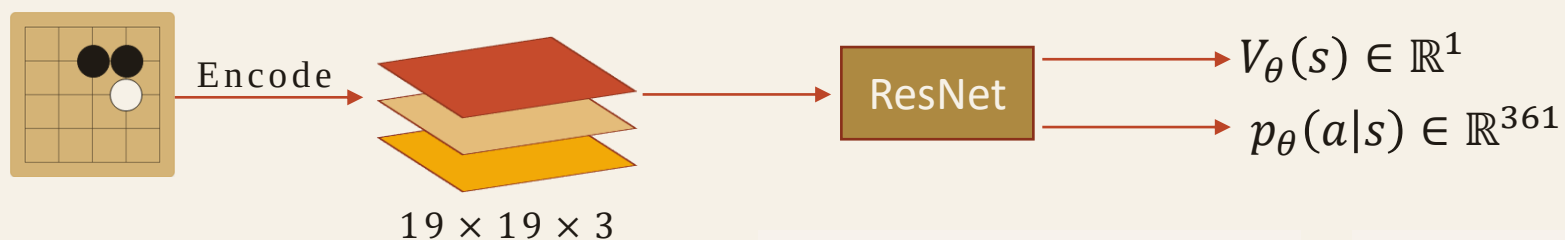
Based on Nash equilibrium, your strategy should be:

$$\pi^* = \operatorname{argmax}_{\pi} \min_{\pi_{opp}} V(\pi, \pi_{opp})$$

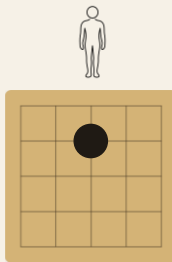
02 Policy Network and Value Network

$$V_{\theta}(s) \rightarrow P(\text{win}) = ?$$

$$p_{\theta}(a|s) \rightarrow$$

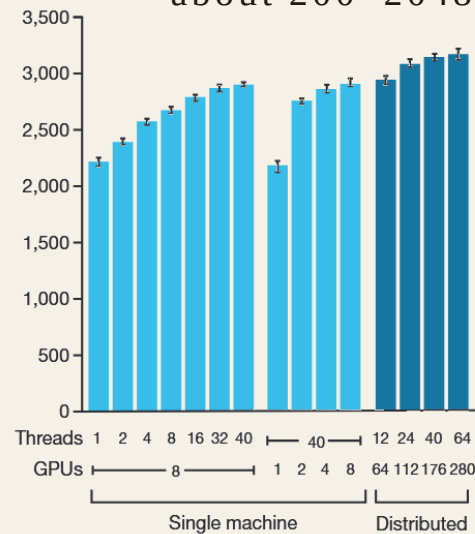



03 Improve MCTS with Neural Network



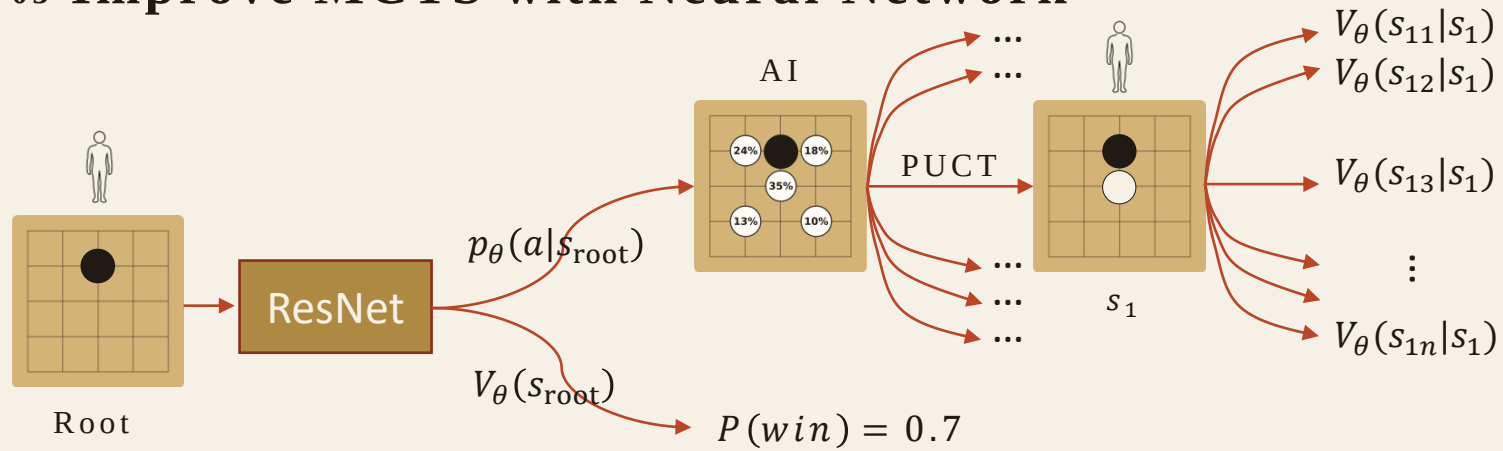
Root

- Number of simulations:
about 200~2048



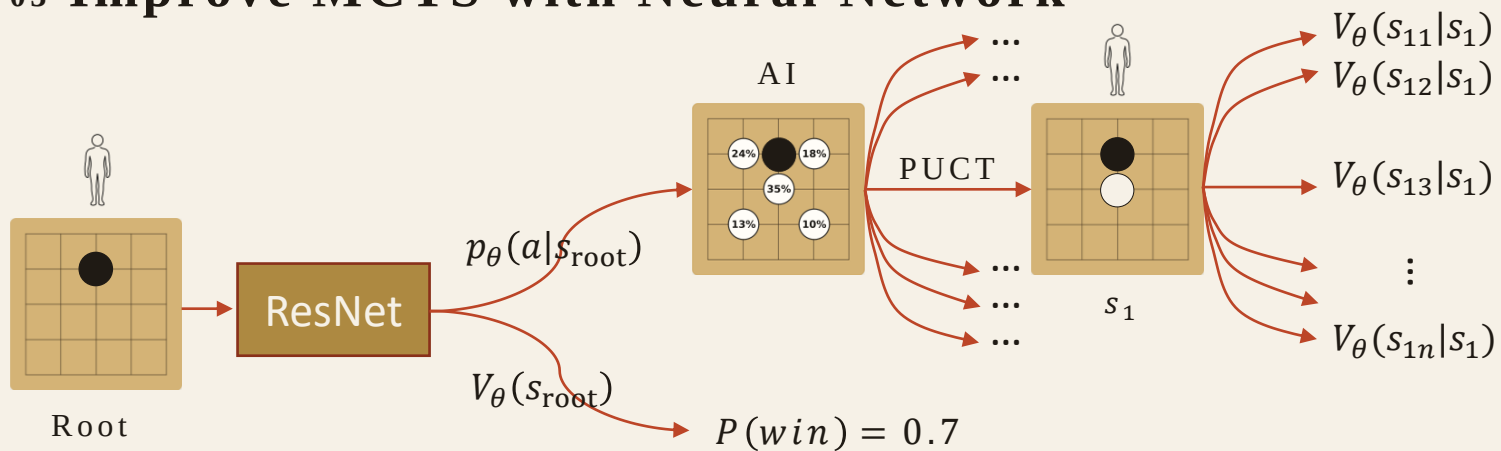
David Silver et al., Nature, 529, 484–489 (2016)

03 Improve MCTS with Neural Network

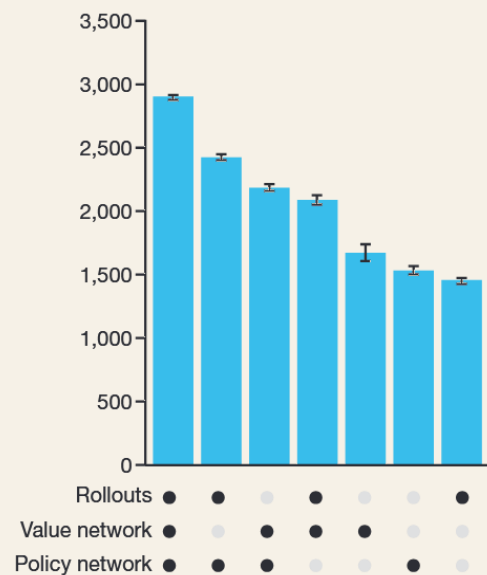


- Number of simulations:
about 200~2048
- Selection $\rightarrow$ PUCT: $\operatorname{argmax}_a \left(Q(s, a) + c_{PUCT} P(s, a) \frac{\sqrt{N}}{1 + N(s, a)} \right)$
- Expansion
- Evaluation
- Backup

03 Improve MCTS with Neural Network

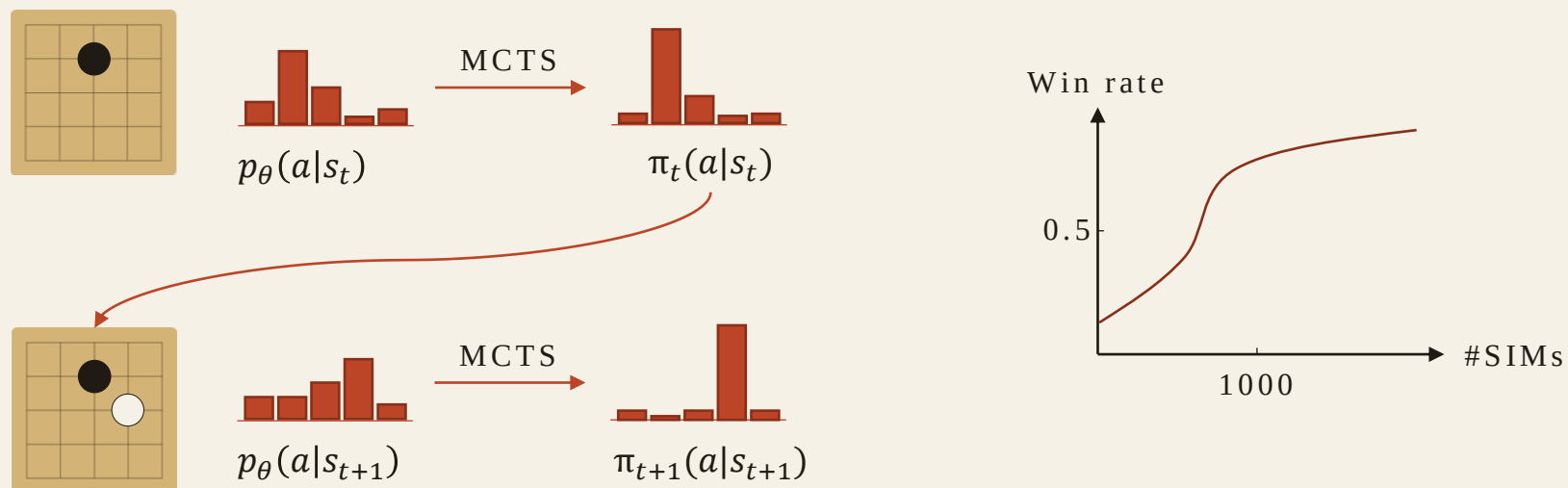


The decision of next move is determined by p_{θ} and V_{θ}



David Silver et al., Nature, 529, 484–489 (2016)

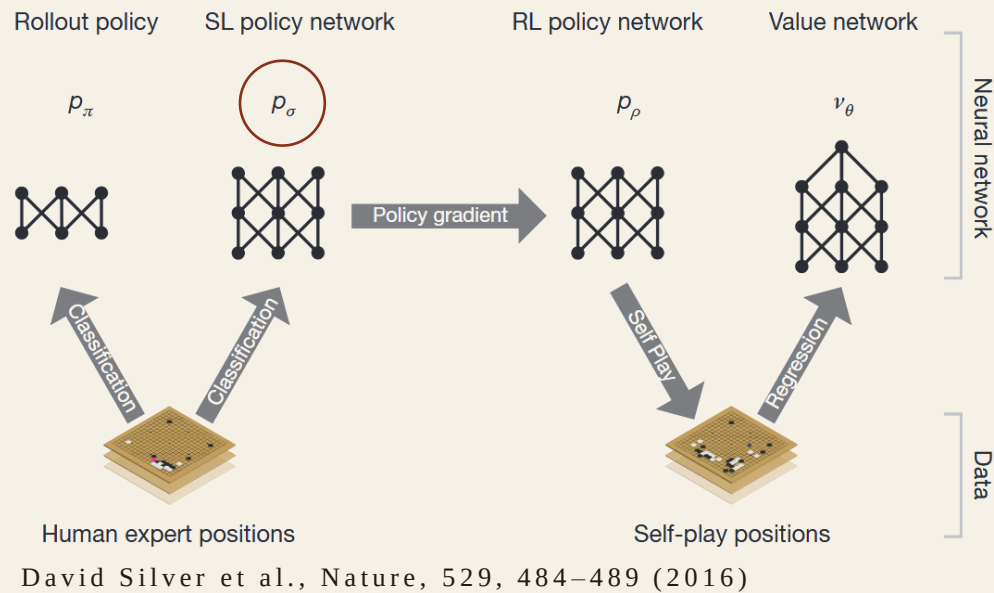
04 Self-play with Reinforcement Learning



Q: Is MCTS surely leads to better results than purely using policy network?

04 Self-play with Reinforcement Learning

In **AlphaGo Lee**, they trained the policy network with supervised learning to initialize.



Supervised learning do:

$$\max_{\sigma} \sum_t \log p_{\sigma}(a_t^{human} | s_t)$$

For reinforcement learning, the reward is:

$$z_t = \begin{cases} +1, & \text{win} \\ -1, & \text{lose} \end{cases}$$

The reinforcement learning loss function for policy network is:

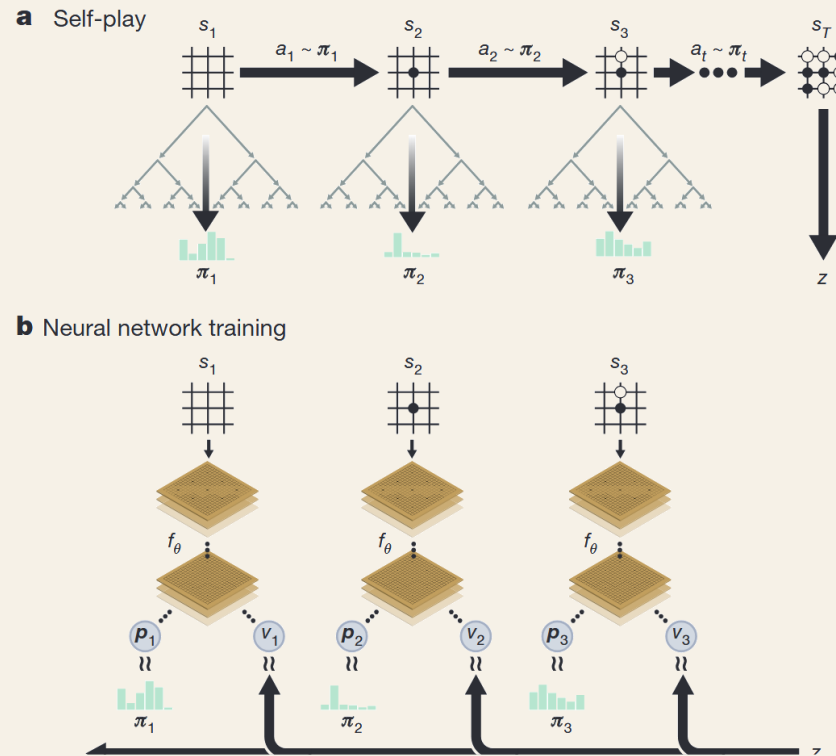
$$\mathcal{L}_p(\rho) = - \sum_t z_t \log p_{\rho}(a_t | s_t)$$

The reinforcement learning loss function for value network is:

$$\mathcal{L}_v(\theta) = (v_{\theta}(s) - z)^2$$

04 Self-play with Reinforcement Learning

In **AlphaGo Zero**, they trained the policy network and the value network *tabula rasa*.



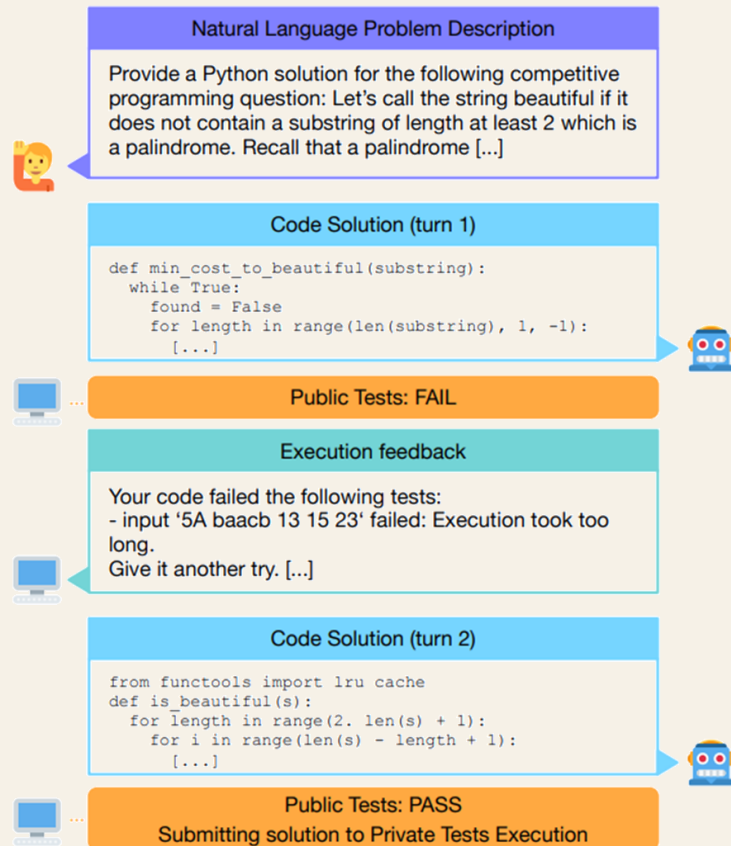
The loss function is:

$$\mathcal{L}(\theta) = (z_t - v_{\theta}(s_t))^2 - \sum_t \pi_t(a|s_t) \log p_{\theta}(a|s_t) + c \|\theta\|^2$$

David Silver et al., Nature 550, 354–359 (2017)

04 Self-play with Reinforcement Learning

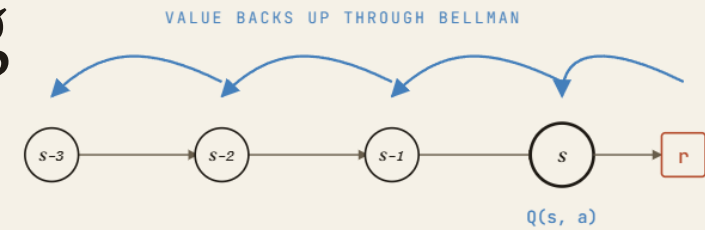
Q: How can simple Reinforcement Learning with Verifiable Rewards (RLVR) produce such complex behavior, while AlphaGo seems less impressive once we understand its engineered MCTS? Do you agree?



Jonas Gehring et al., <https://arxiv.org/abs/2410.02089>

PART 03 • OTHER REINFORCEMENT LEARNING APPROACHES

03 · Other Reinforcement Learning Approaches



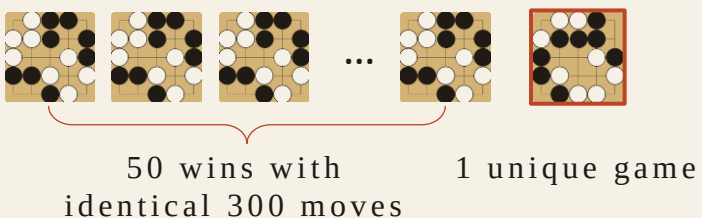
01 Can LLM-style reinforcement learning be applied to Go?



An extreme example:

$$\pi_A = \pi_B$$

A beat B with 51:49 win-lose.

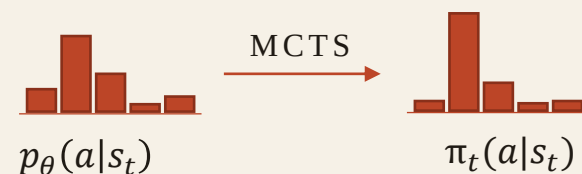


The LLM-style gradient is:

$$\hat{g} = R \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$$

Why MCTS can improve this?

$$(p_t, v_t) = f_{\theta}(s_t) \rightarrow \text{MCTS} \rightarrow \pi_t$$



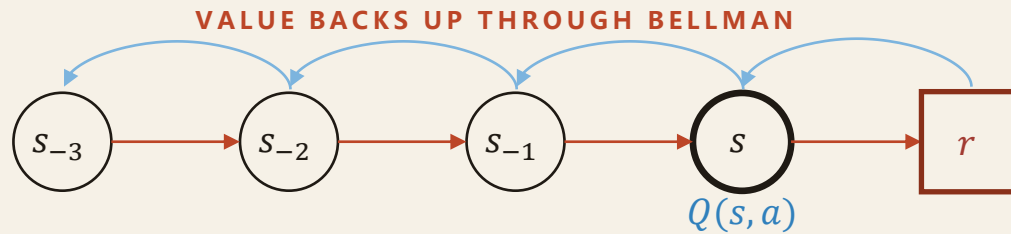
The loss function is:

$$\hat{g} = 2(z - v) \nabla_{\theta} v + \sum_a \pi_a \nabla_{\theta} \log p_a - 2c\theta$$

02 Neural Fictitious Self-Play (NFSP)



02 Neural Fictitious Self-Play (NFSP)



Train π_A to against **a fixed strategy** π_B :

$$\pi_A \leftarrow \text{BR}(\pi_B)$$

Train on different opponent policies:

$$\pi_A \leftarrow \text{BR} \left(\sum_j \mu_j \pi_j \right)$$

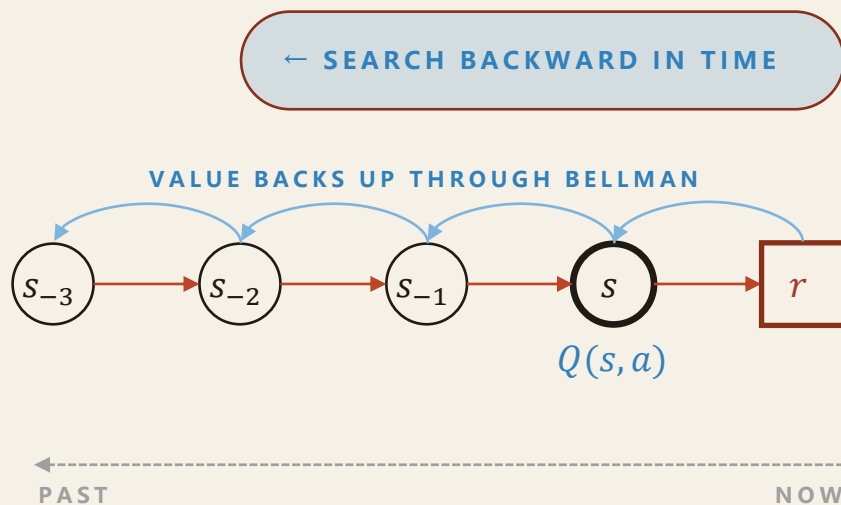
BELLMAN TARGET AT S

$$Q(s_t, a_t) \leftarrow r_t + \gamma \max_{a'} Q(s_t, a')$$

03 NFSP vs. MCTS

TEACHER A

NFSP • Best-response via TD

Q-learning along observed trajectories

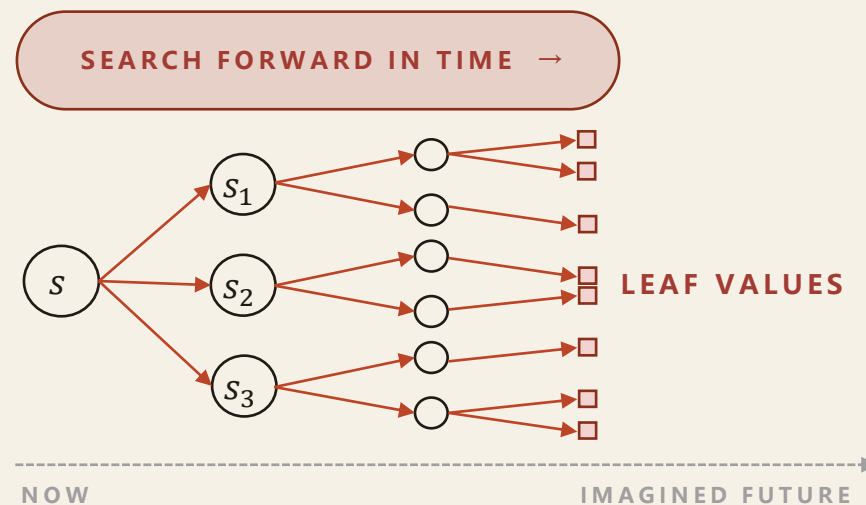
BELLMAN TARGET AT S

$$Q(s_t, a_t) \leftarrow r_t + \gamma \max_{a'} Q(s_t, a')$$

Teacher = greedy a . Better actions discovered in retrospect through a neural network or a Q table.

TEACHER B

MCTS • Search via simulated rollouts

UCT expansions through a learned model

BACKED-UP VALUE AT S

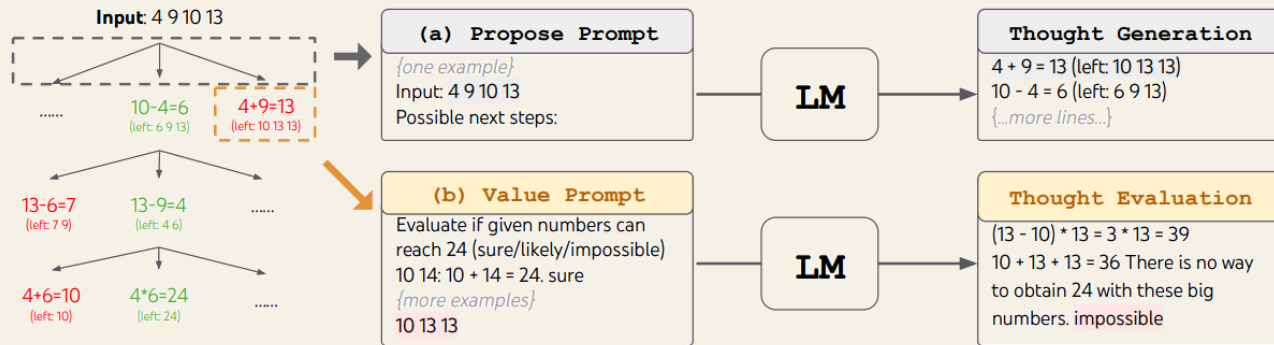
$$Q(s, a) \approx \mathbb{E}_{\tau \sim \text{tree}} [\sum_t \gamma^t r_t]$$

Teacher = *visit counts*. Better actions discovered in foresight.

Both teachers find a better action at s for the student π_θ to imitate — NFSP from trajectories that **already happened**, MCTS from trajectories that **haven't**. The student doesn't know the difference.

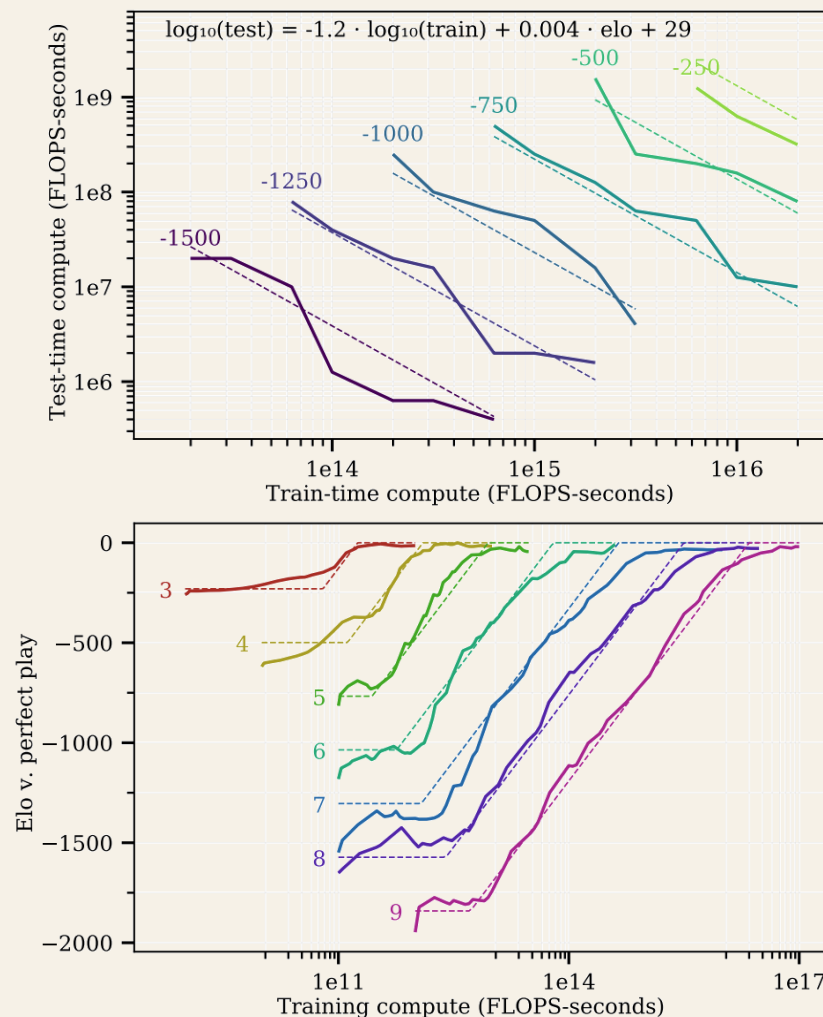
04 Why MCTS is currently not suitable for LLM?

Standard PUCT is not compatible to LLM.

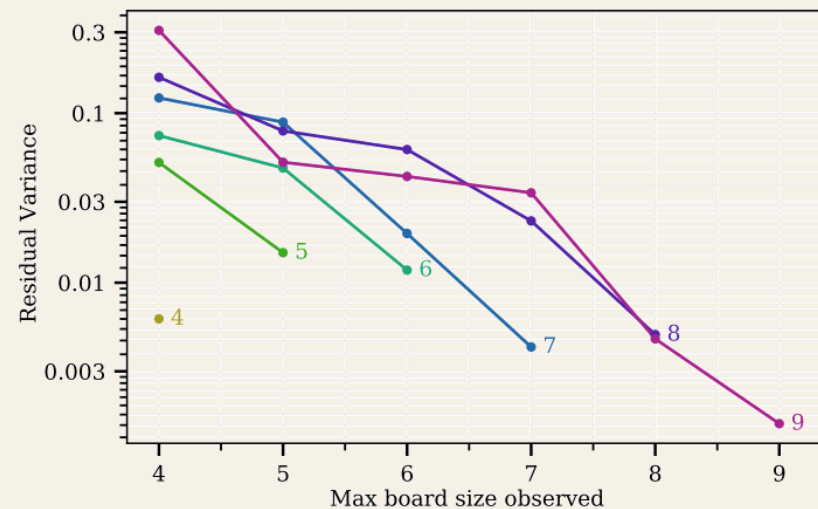


Shunyu Yao et al., arXiv:2305.10601v2

05 Train-time scaling and test-time scaling



FLOPS $\approx f(\text{board size, target Elo})$



The Bitter Lesson

Rich Sutton

March 13, 2019

https://www.cs.utexas.edu/~eunsol/courses/data/bitter_lesson.pdf

PART 04 • From Sparse Rewards to Dense Supervision

04 • How Search Improves RL Efficiency



01 bits per FLOP

$$\frac{\text{bits}}{\text{FLOP}} = \frac{\text{Samples}}{\text{FLOP}} \times \frac{\text{bits}}{\text{sample}}$$

LLM: The sky is ...

Vocab size = 100,000, correct answer is “blue”.

$$I_{SL} = -\log_2 p$$

If $p = 10^{-5}$, $I_{SL} = 16.6$ bits.

However, in naive RL, the entropy is

$$H_2(p) = -p \log_2 p - (1 - p) \log_2 (1 - p)$$

If $p = 10^{-5}$, $H_2(p) = 0.00018$ bits.

02 Why Is Iterative Search So Much More Powerful Than LLM Reinforcement Learning?

One-hot: $y = \{1, 0, 0, 0\}$;

MCTS: $\pi = \{0.52, 0.36, 0.10, 0.02\}$

Assume LLM outputs a sequence

$$y = (y_1, \dots, y_T)$$

The autoregression probability is

$$p_{\theta}(y|x) = \prod_{t=1}^T \pi_{\theta}(y_t|x, y_{<t})$$

The score function of each token

$$S_t = \nabla_{\theta} \log \pi_{\theta}(y_t|x, y_{<t})$$

Then

$$\nabla_{\theta} \log p_{\theta}(y|x) = \sum_{t=1}^T S_t$$

Case A: Only have a single terminal reward

If we take the entire answer as an action

$$\hat{g}_{single} = R(y) \sum_{t=1}^T S_t$$

If we take each token as an separate action

$$\hat{g}_{multi} = \sum_{t=1}^T G_t S_t$$

Because we only have a single terminal reward

$$r_1 = \dots = r_{T-1} = 0, \quad r_T = R$$

For each t

$$G_t = R$$

$$\hat{g}_{single} = \hat{g}_{multi} R(y) \sum_{t=1}^T S_t$$

02 Why Is Iterative Search So Much More Powerful Than LLM Reinforcement Learning?

Case B: If reliable per-step rewards are available

For single-step RL

$$\hat{g}_{single} = \sum_{t=1}^T \left(\sum_{k=1}^T r_k \right) S_t$$

For multi-step RL

$$\hat{g}_{multi} = \sum_{t=1}^T \left(\sum_{k=t}^T r_k \right) S_t$$

The difference is

$$\hat{g}_{multi} - \hat{g}_{single} = \sum_{t=1}^T \left(\sum_{k < t} r_k \right) S_t$$

$$\text{Var}(\hat{g}_{multi}) < \text{Var}(\hat{g}_{single})$$

Where comes the high variance of LLM RL?

$$\hat{g} = R(y) \sum_{t=1}^T S_t$$

The variance is

$$\text{Var} \left(R(y) \sum_{t=1}^T S_t \right) = \sum_t \text{Var}(RS_t) + 2 \sum_{i < j} \text{Cov}(RS_i, RS_j)$$

The length of covariance term is

$$\frac{T(T-1)}{2}$$

Therefore, in the worst case, the variance scales quadratically with the sequence length

$$O(T^2)$$

Credit assignment problem!

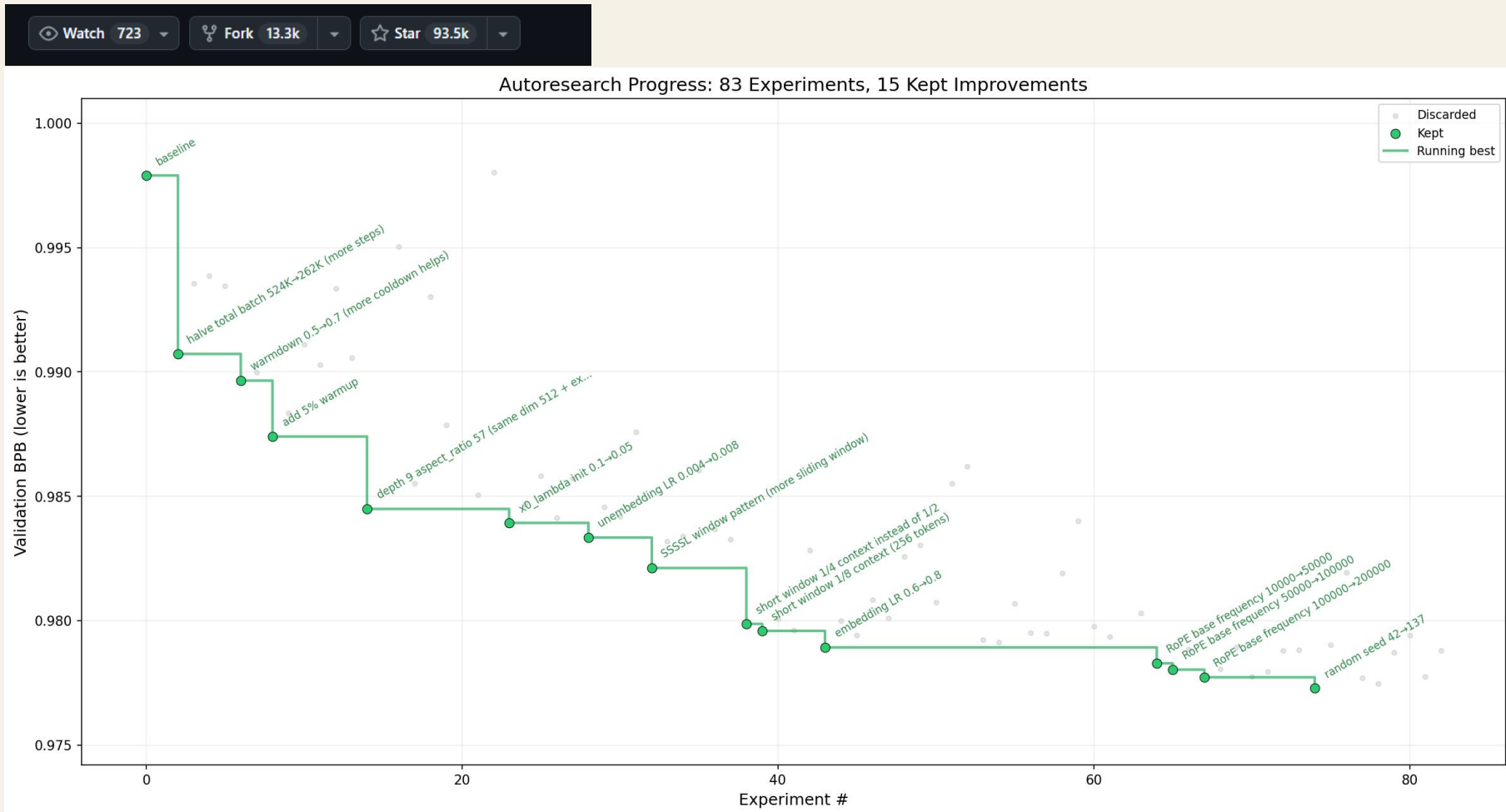
PART 05 • Building a strong Go AI from scratch with modern AI tools.

05 · Automating Go Research with modern LLMs



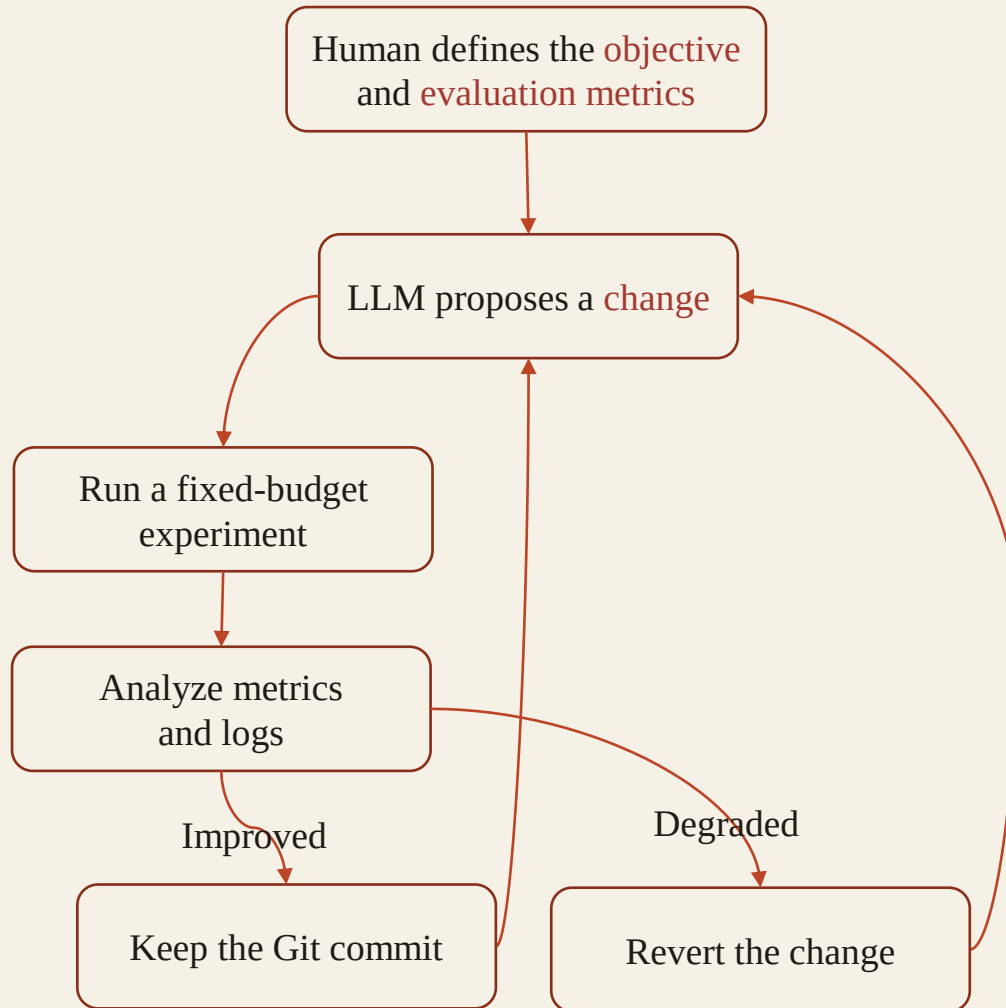
01 What Is AutoResearch?

Karpathy published a skill called AutoResearch in March 2026.



[karpathy/autoresearch: AI agents running research on single-GPU nanochat training automatically](#)

01 What Is AutoResearch?



Learning rate
Weight decay
Batch size
Optimizer
Loss formulation
Seed
...

02 From hyperparameter tuning to open-ended program searching

Traditional hyperparameter optimization is predefined by human expert

$$\lambda = (\textit{learning rate}, \textit{weight decay}, \textit{depth} \dots)$$

Algorithms can only search under the given searching space

$$\lambda^* = \underset{\lambda}{\operatorname{argmin}} L(\lambda)$$

The behavior space of LLMs is not a set of numbers but **the whole program space**

LLMs is a great executer but not a leader

Inner loop

- Coding
- Training
- Debug
- Optimize GPU utilization
- ...

Outer loop

- Long term progress
- Is the Go AI get stronger?
- Predict the scaling law profile?
- ...

03 A customized skill for Go experiment

 **AutoGo Experiment** 我想最大化实验的吞吐量。实验在GPU2-5上进行，请你帮我研究一下如何设置 leaf batchsize和training阶段的batchsize，所有代码做最小改动。

+  Approve for me

5.6 Sol Max ▾



1. Create a self-contained experiment folder **with date and descriptive short name**.
2. Create a .py file that runs the whole **experiment pipeline**.
3. Save **artifacts and data** to /artifacts and /data subfolders follow each experiment.
4. Review the experiment results and conclude what is known and what is unknown.
5. Write a **report.md** to summarize.

04 Build Go AI from scratch

1. From algorithms to a system

Rules + MCTS + Neural network $\neq$ AutoGo

AutoGo=

Tested Code

- + Self-Generated Data
- + Training Loop
- + Objective Evaluation
- + Experiment Control

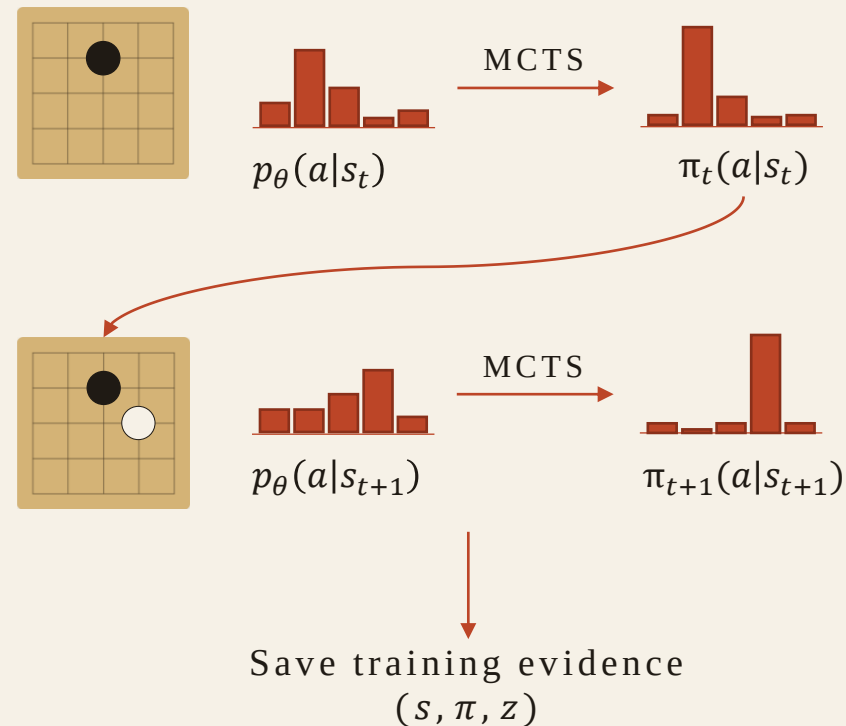
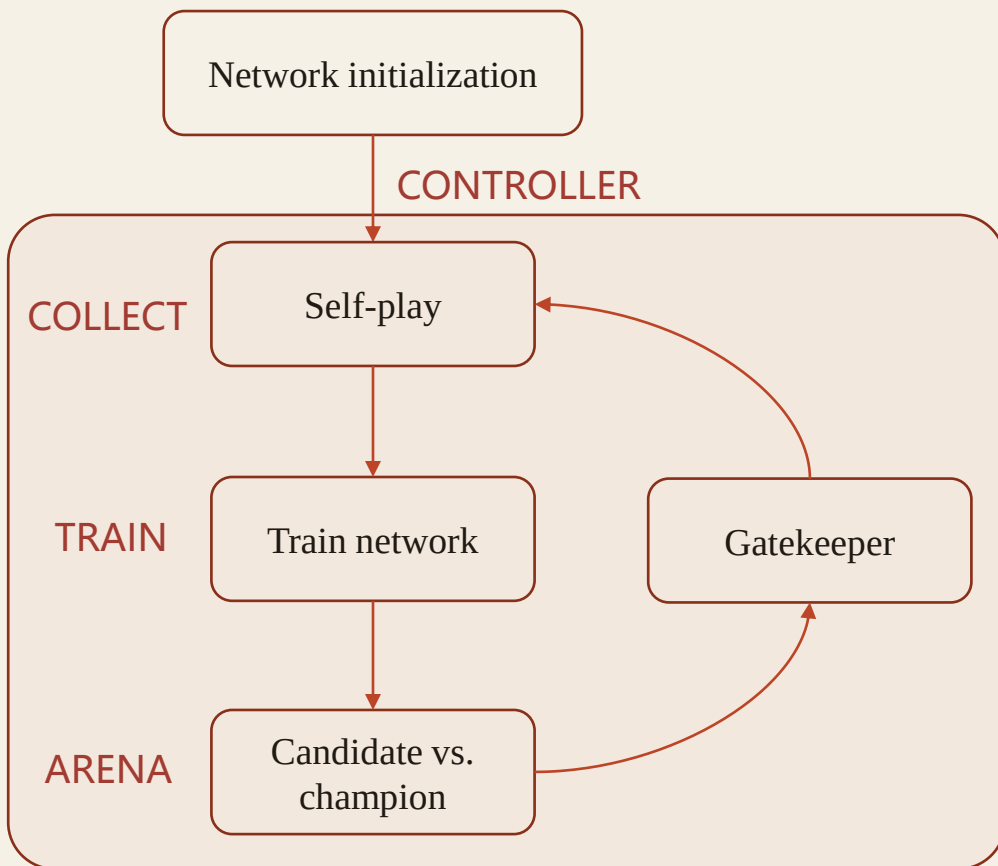
2. Organize the code

src/
experiments/<name>/

```
autogo/  
├─ src/alpha_go/  
│   ├── go.py           # board state + rules  
│   ├── model.py        # fθ(s) → policy, value  
│   ├── mcts.py         # search improves policy  
│   ├── agents/nn_mcts.py # model + MCTS player  
│   ├── gameplay.py     # play and record games  
│   ├── self_play.py    # generate NPZ data  
│   └─ dataset.py       # NPZ → training samples  
├─ experiments/ppt-demo/  
│   ├── config.json     # frozen hypothesis  
│   ├── controller.py   # experiment state machine  
│   ├── train.py  
│   ├── arena.py  
│   ├── status.py  
│   └─ runtime/         # state, events, logs  
└─ tests/               # executable specifications
```

04 Build Go AI from scratch

3. Close the learning loop



EXPLORE

Dirichlet noise + temperature

SUPERVISE

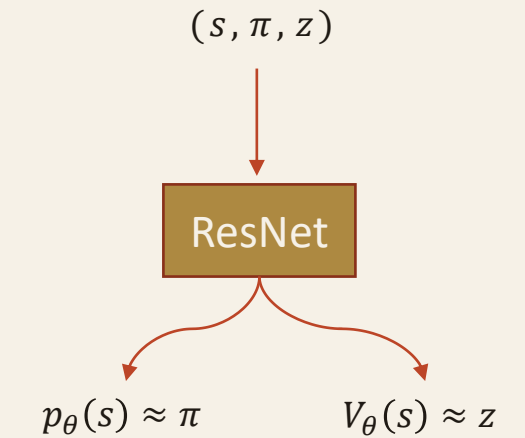
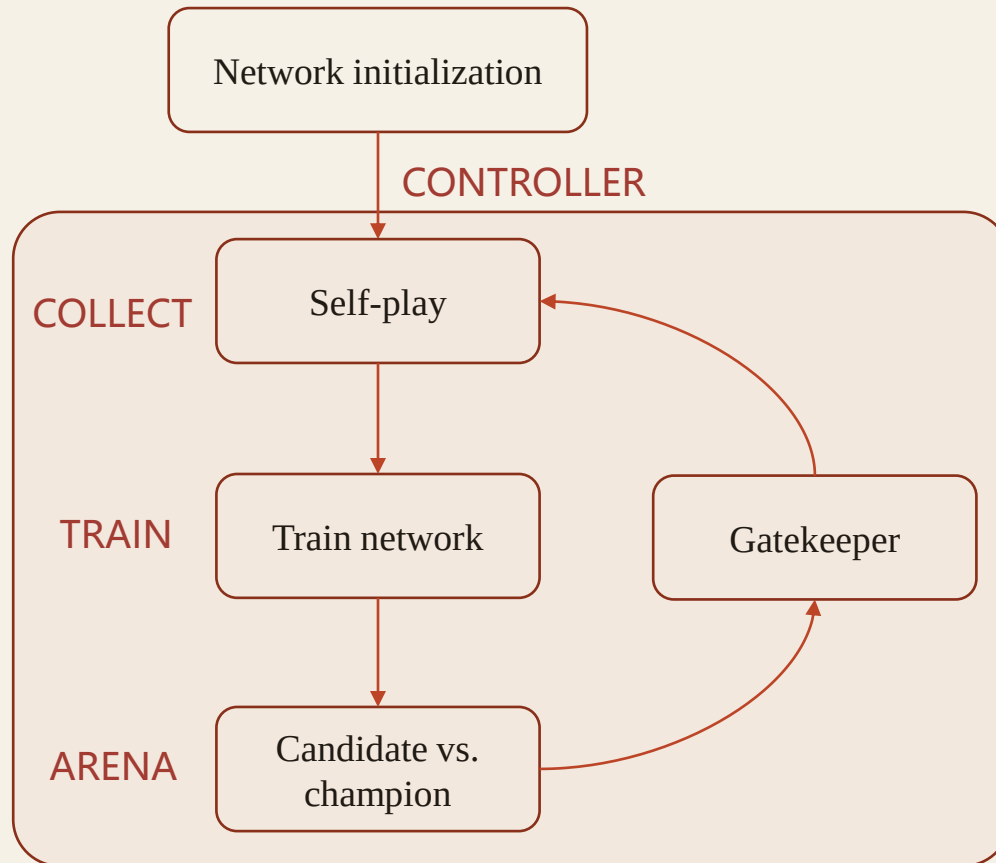
MCTS visits + game outcome

MONITOR

diversity + early pass + duplicates

04 Build Go AI from scratch

3. Close the learning loop



Thousands of search evaluations
↓ **distillation**
One neural-network forward pass

POLICY

Imitate the searched distribution

VALUE

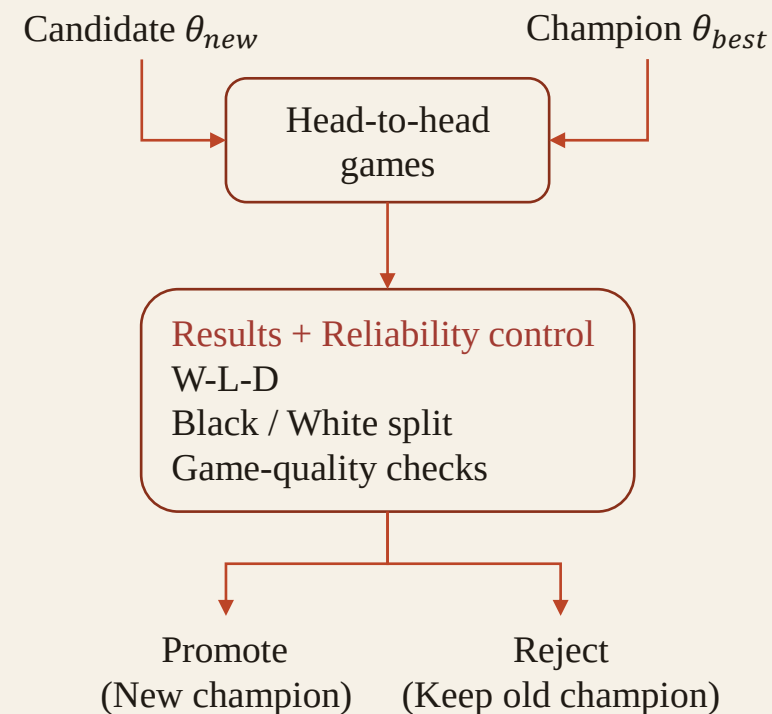
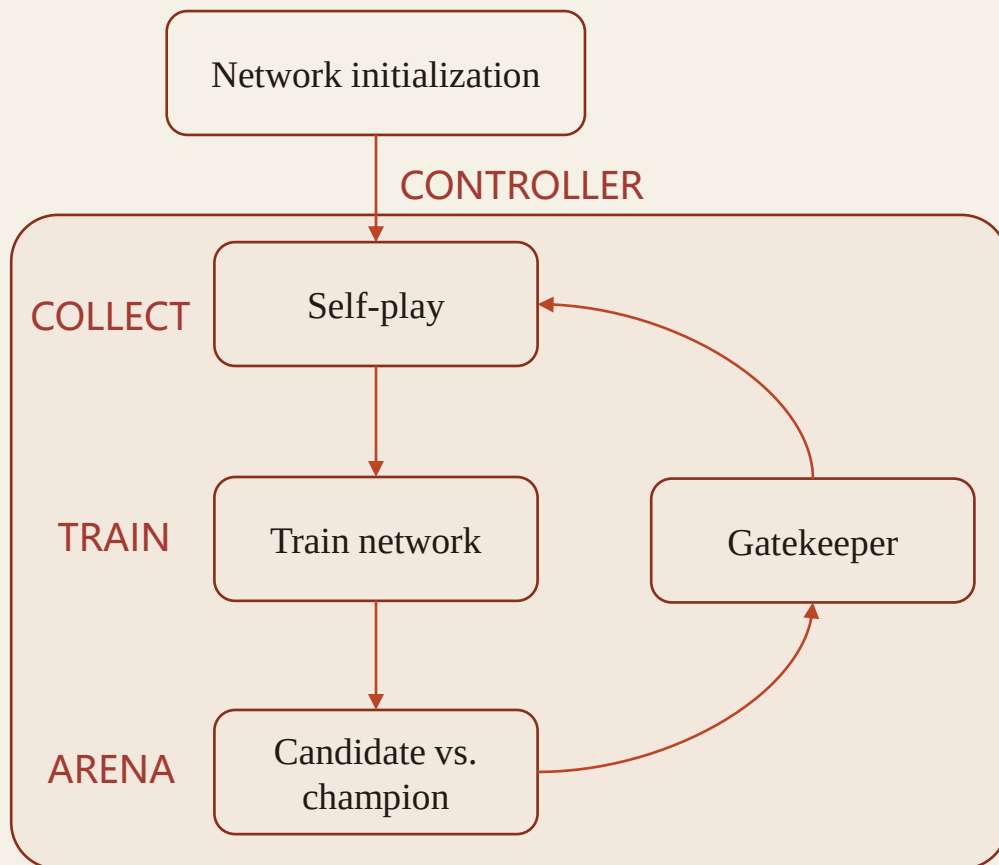
Predict the final outcome

GENERALIZE

Improve unseen positions

04 Build Go AI from scratch

3. Close the learning loop



CONTROL

Frozen champion + equal budget

MEASURE

W-L-D + black/white split

DECIDE

Promote or reject

04 Build Go AI from scratch

4. Auto experiment with LLMs

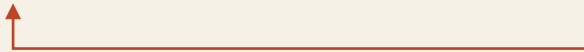
Human

defines goal, constraints, success criteria



LLM + Tools

Inspect → Edit → Test → Run → Observe → Diagnose



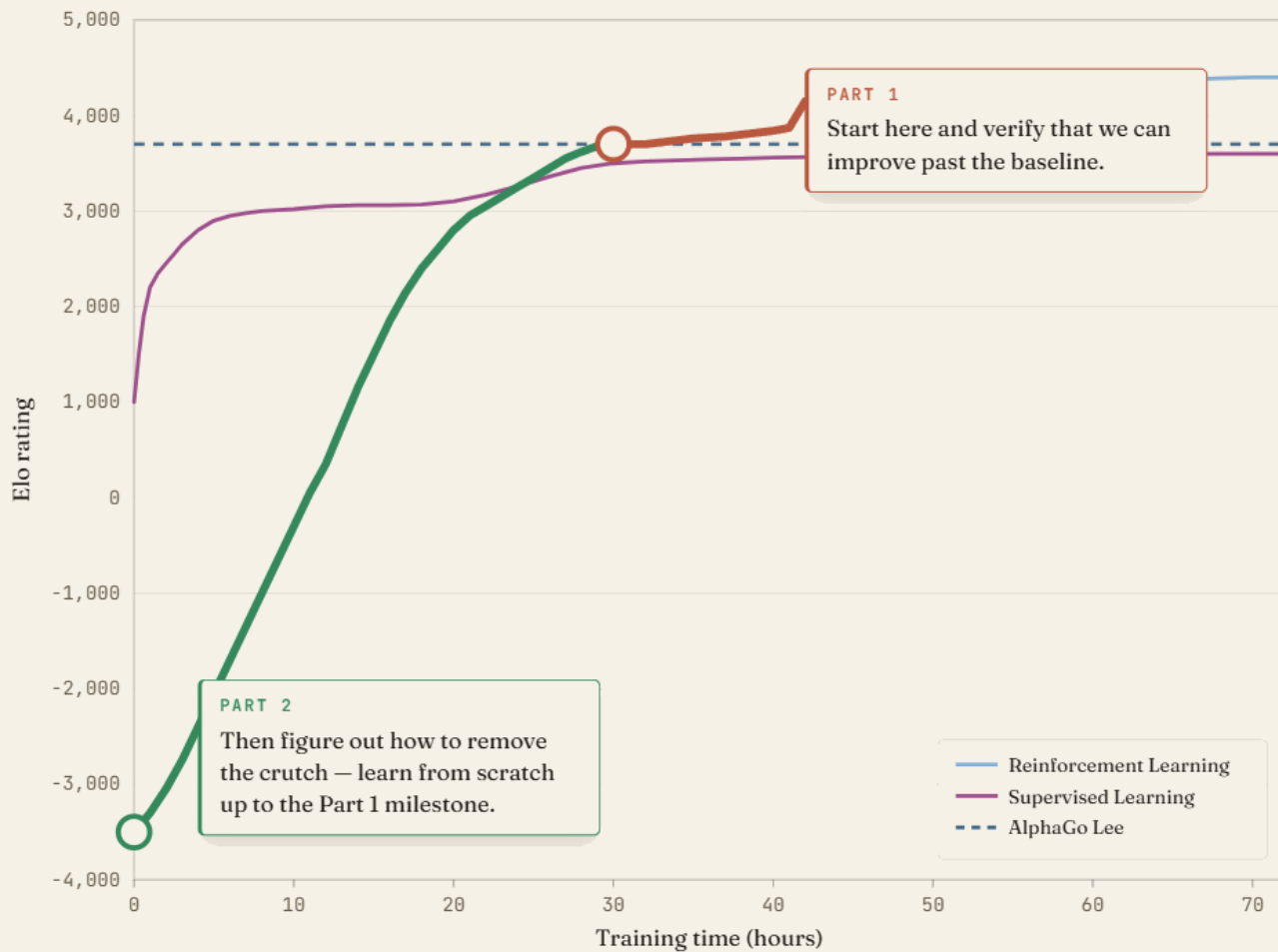
05 Build Go AI from scratch (experiments)

Create a skill to plot results in certain styles

 **Skill Creator** 我现在想创建一个skill，叫做AlphaGo-ppt-graph，请你看一下我的ppt里的图片的风格，总结一下，包括字体，配色等，背景要求透明。我希望我调用这个skill的时候能够让AI制作的plot跟我ppt里已有的图片风格接近。（我说的图片不是cite别人文章里的图片，而是一些自己画的或者你帮我之前画的图表等，引用的图片底下我会添加引用来源的）

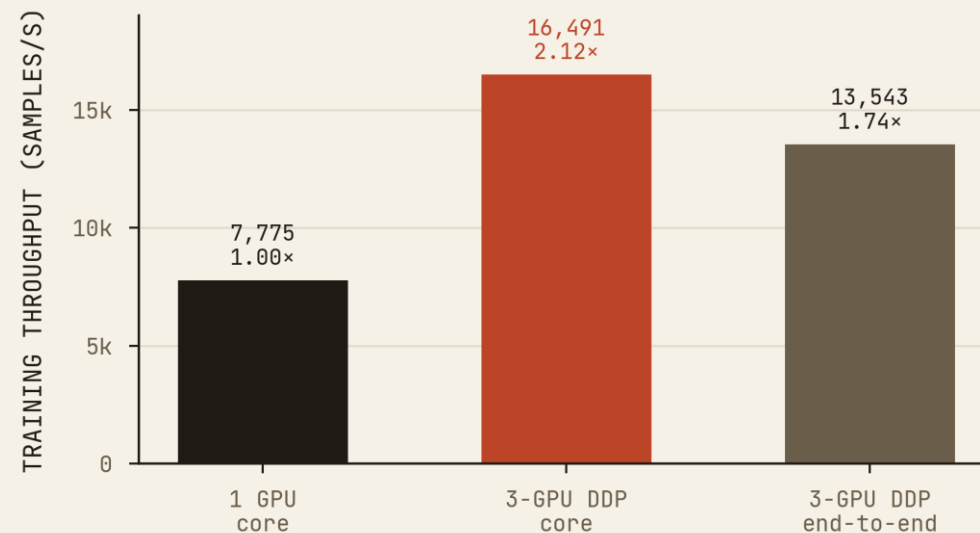
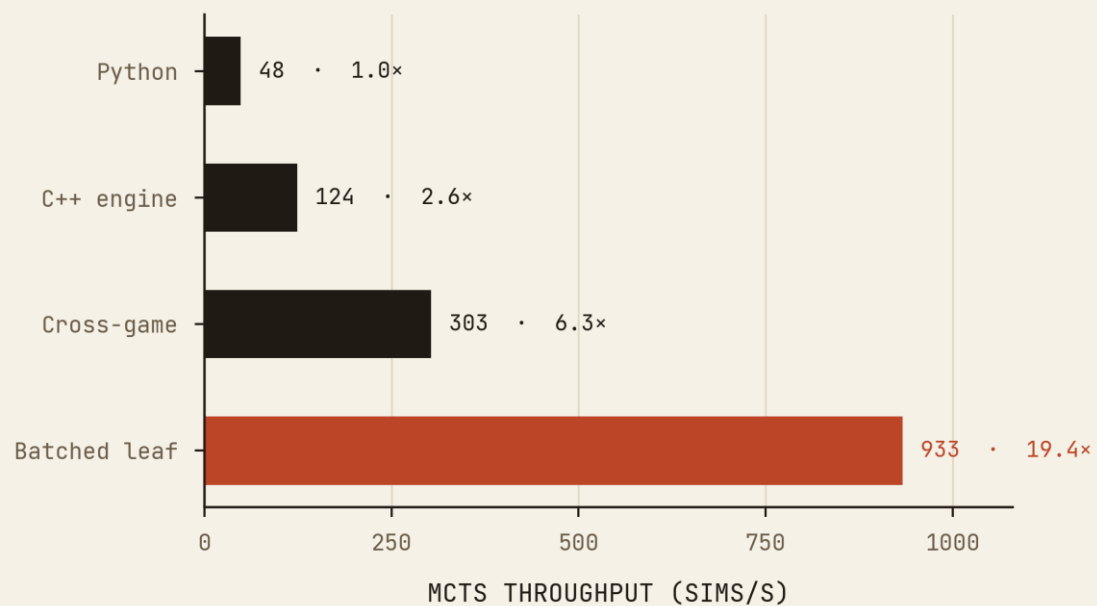
05 Build Go AI from scratch (experiments)

1. Start from a good init (against KataGo)



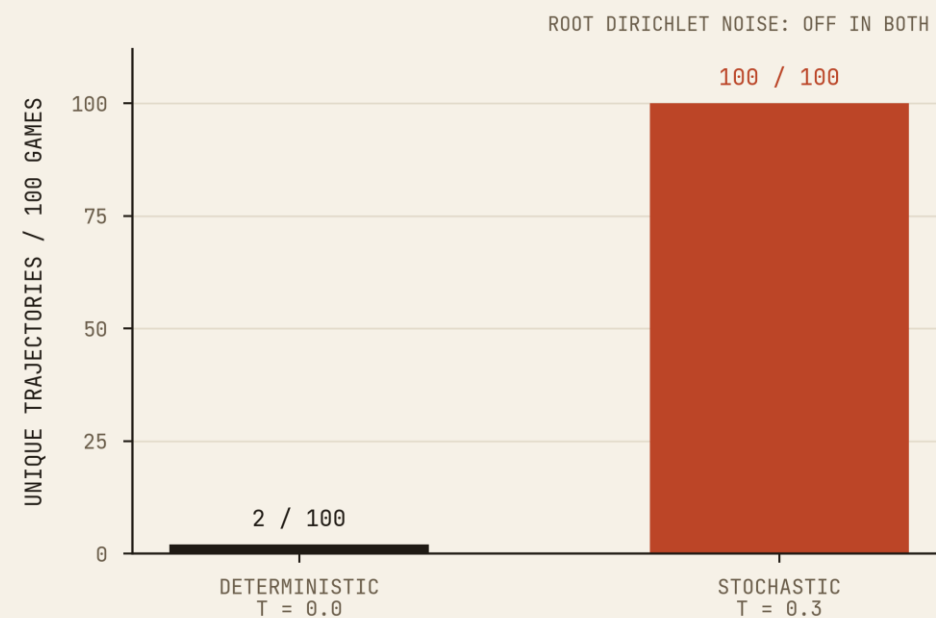
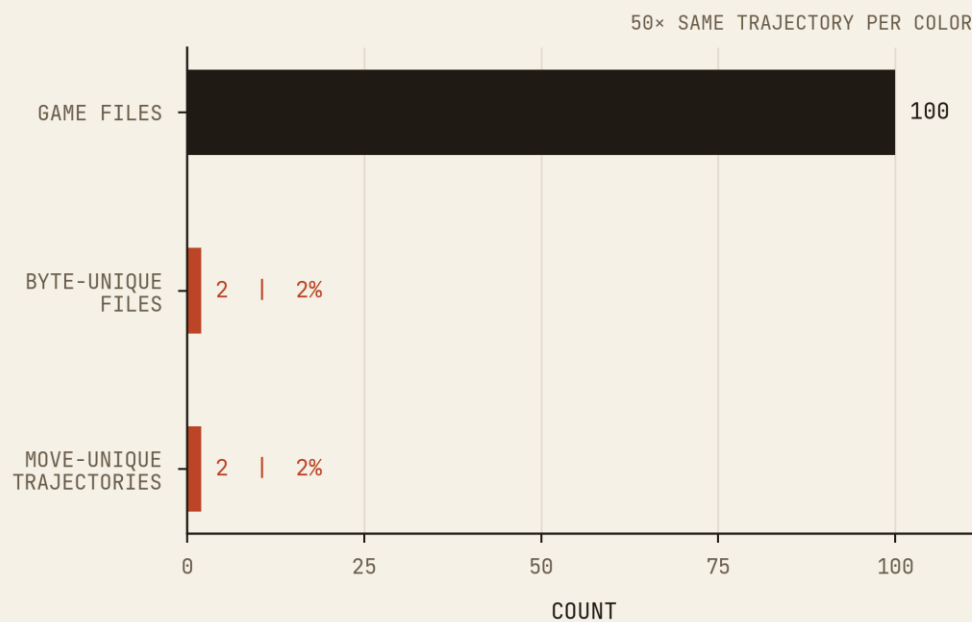
05 Build Go AI from scratch (experiments)

2. Acceleration



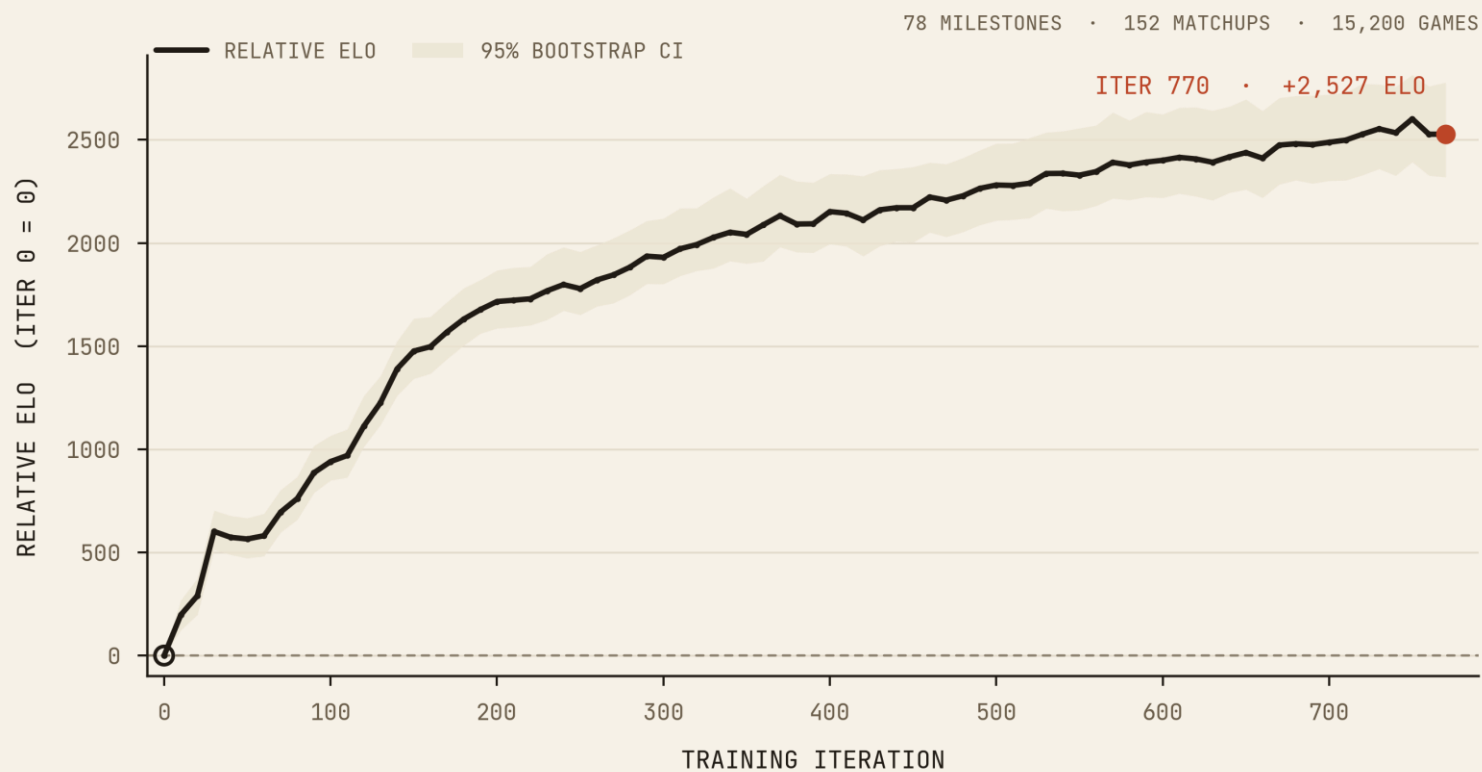
05 Build Go AI from scratch (experiments)

3. Diversity of samples



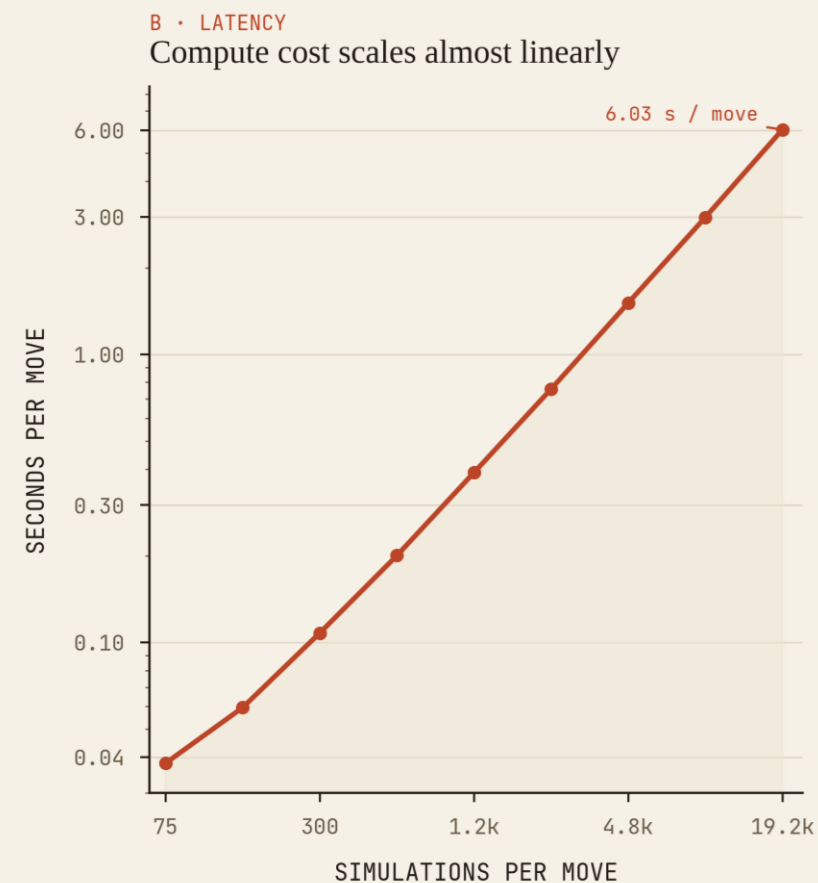
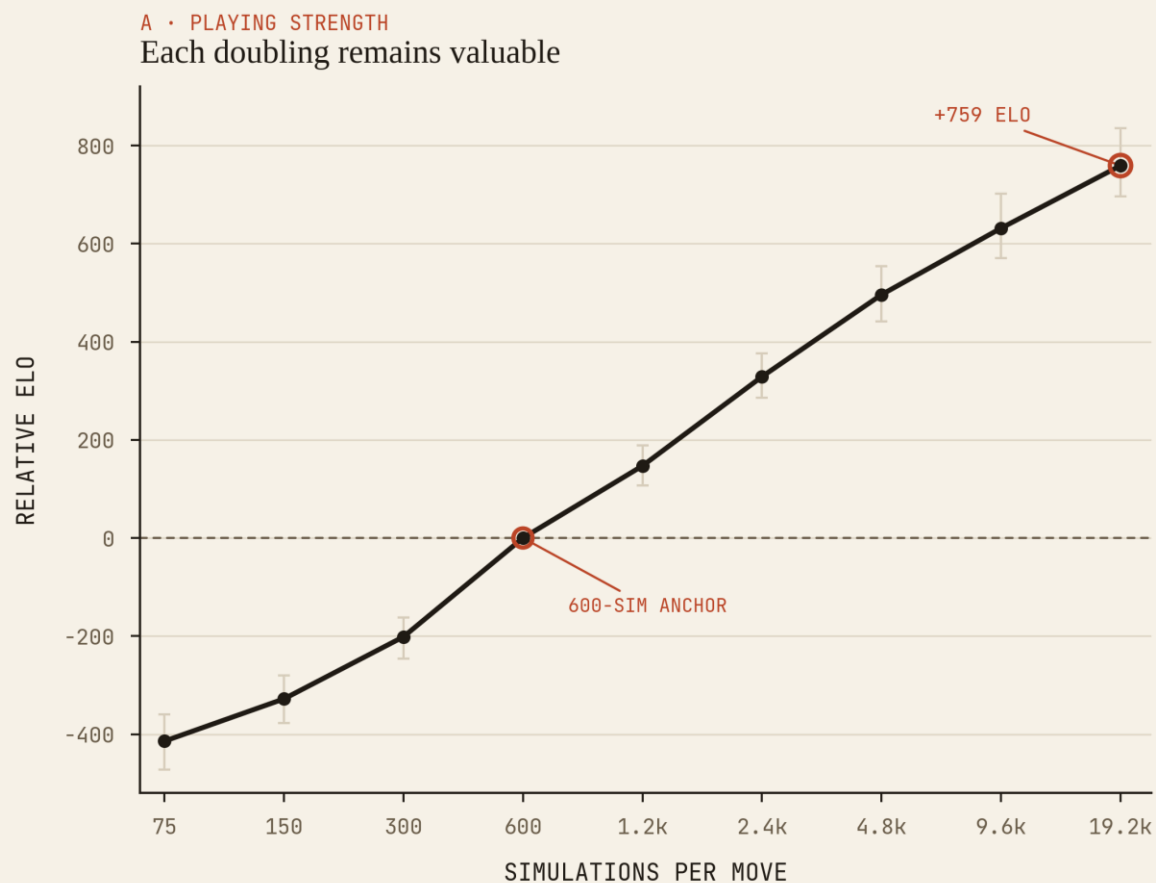
05 Build Go AI from scratch (experiments)

4. Elo rating



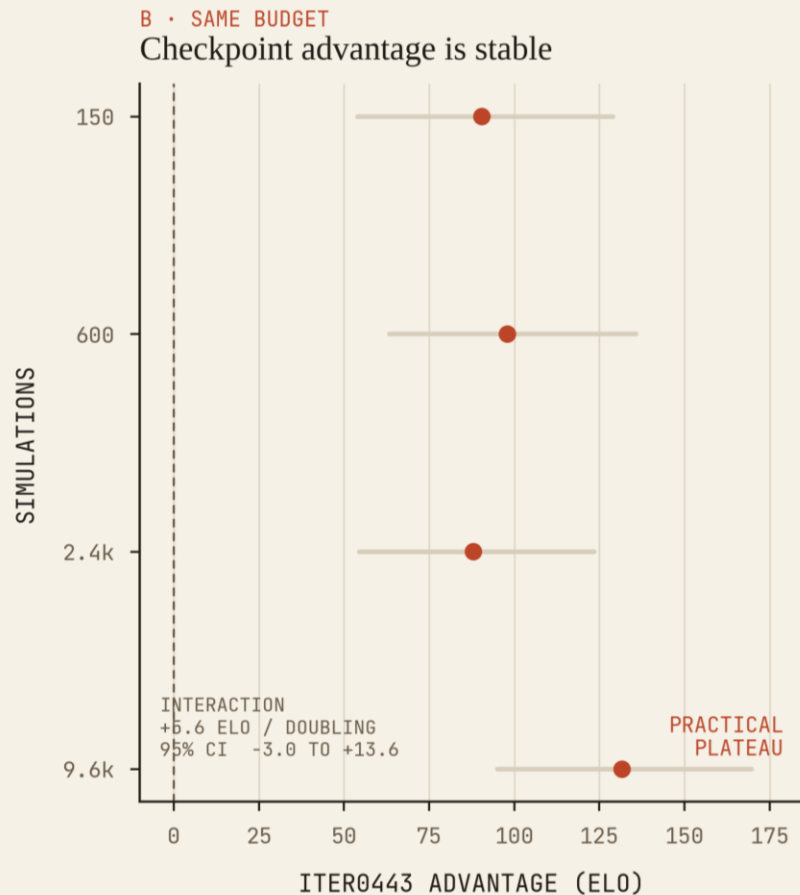
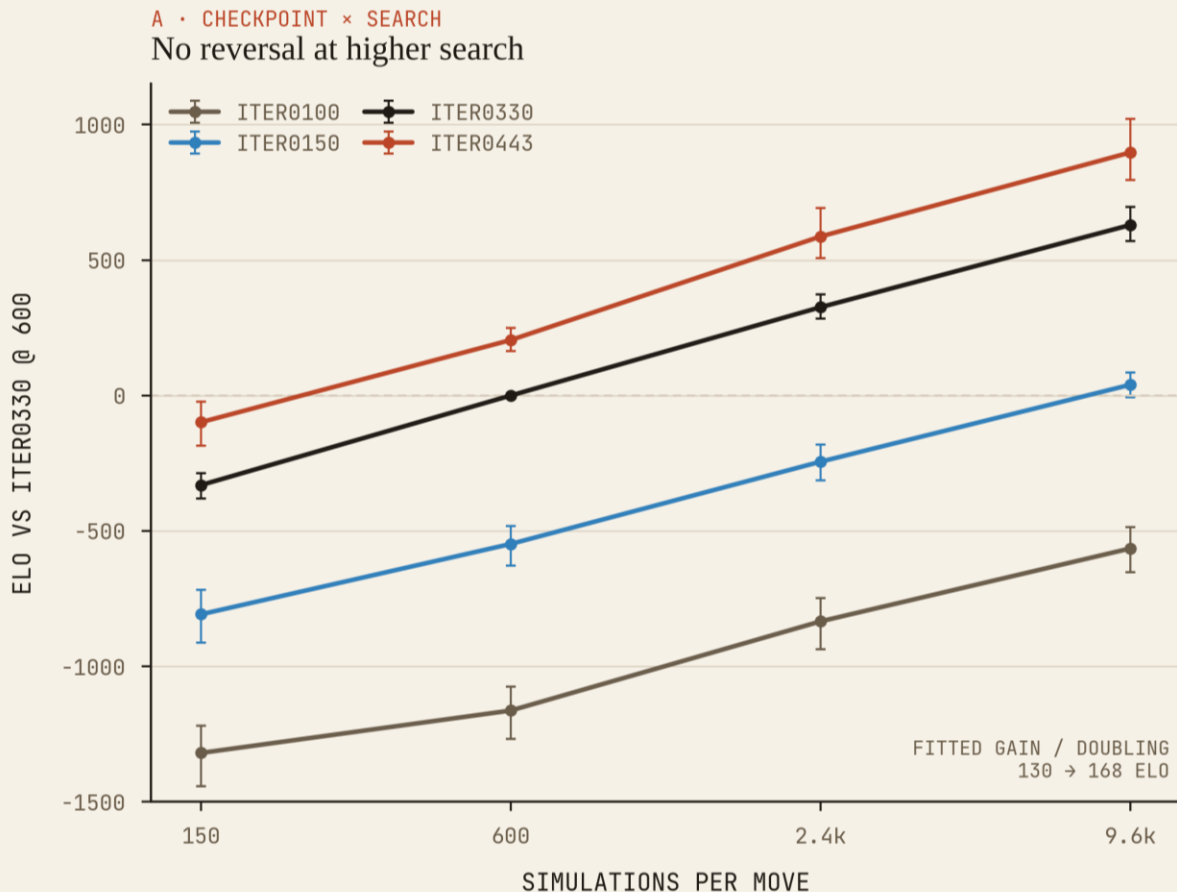
05 Build Go AI from scratch (experiments)

5. MCTS scaling and computing budget



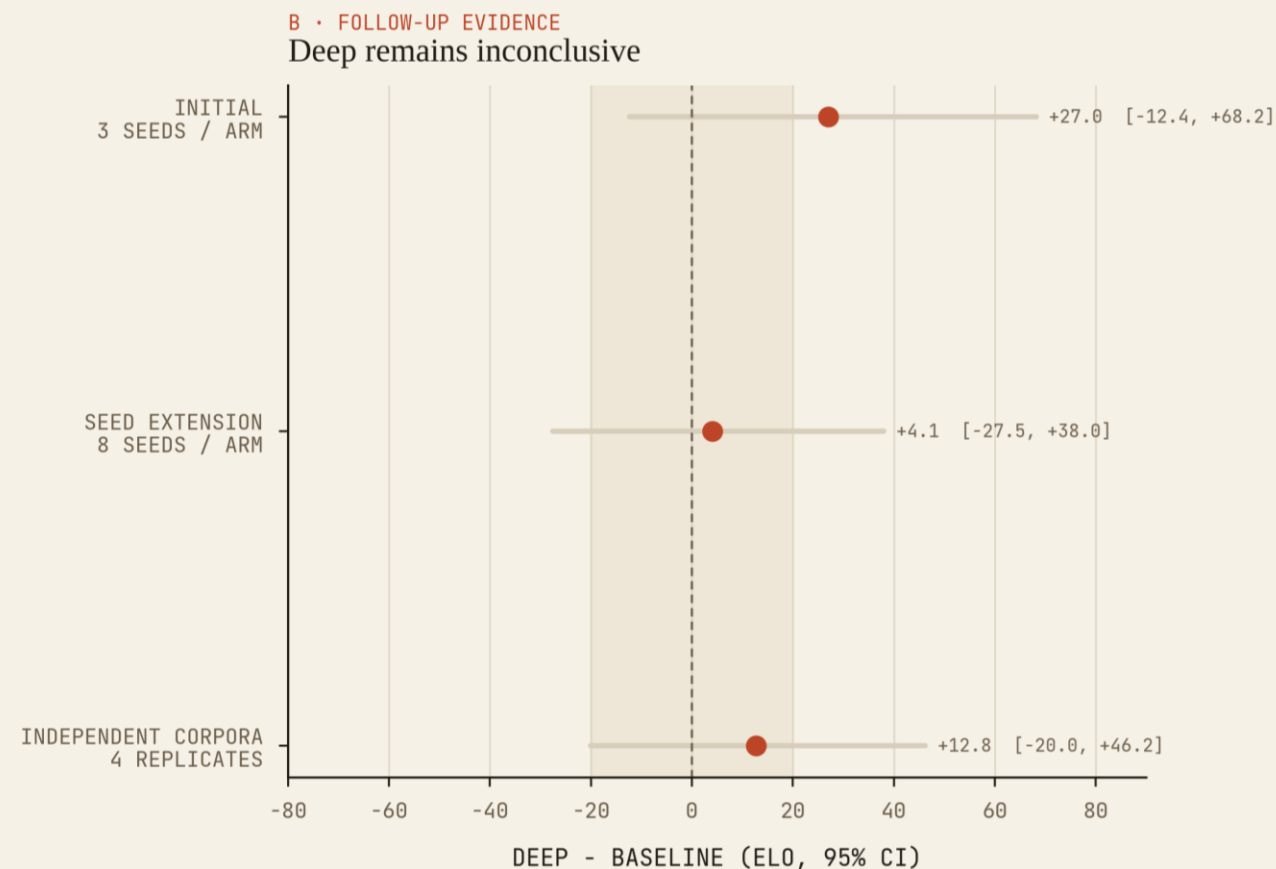
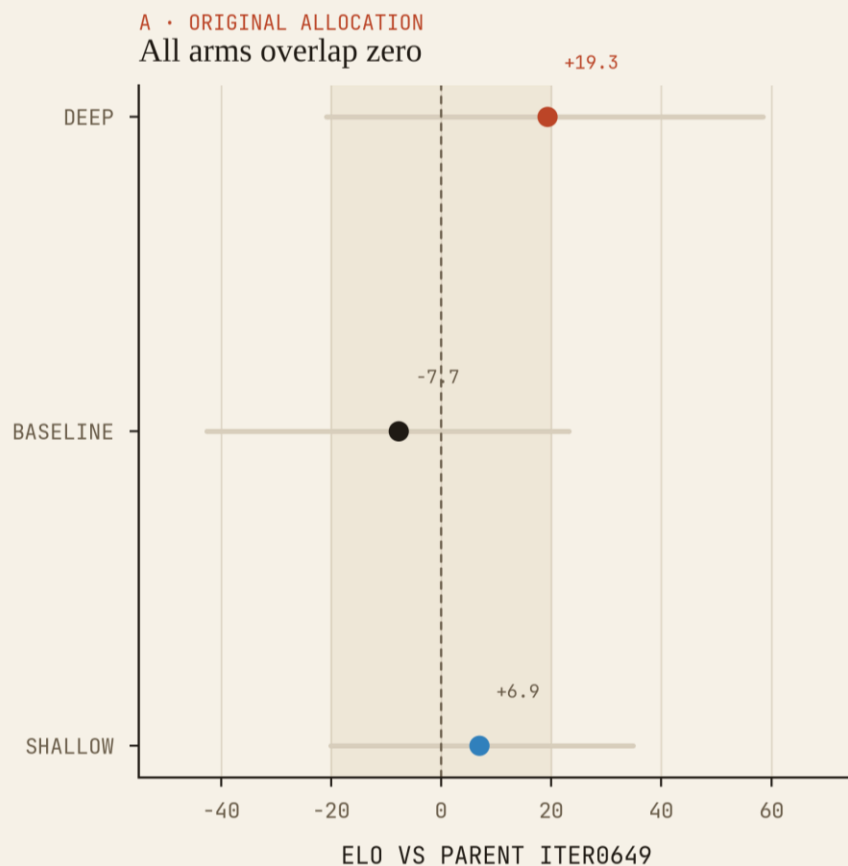
05 Build Go AI from scratch (experiments)

6. The advantage of MCTS is amortized into neural networks



05 Build Go AI from scratch (experiments)

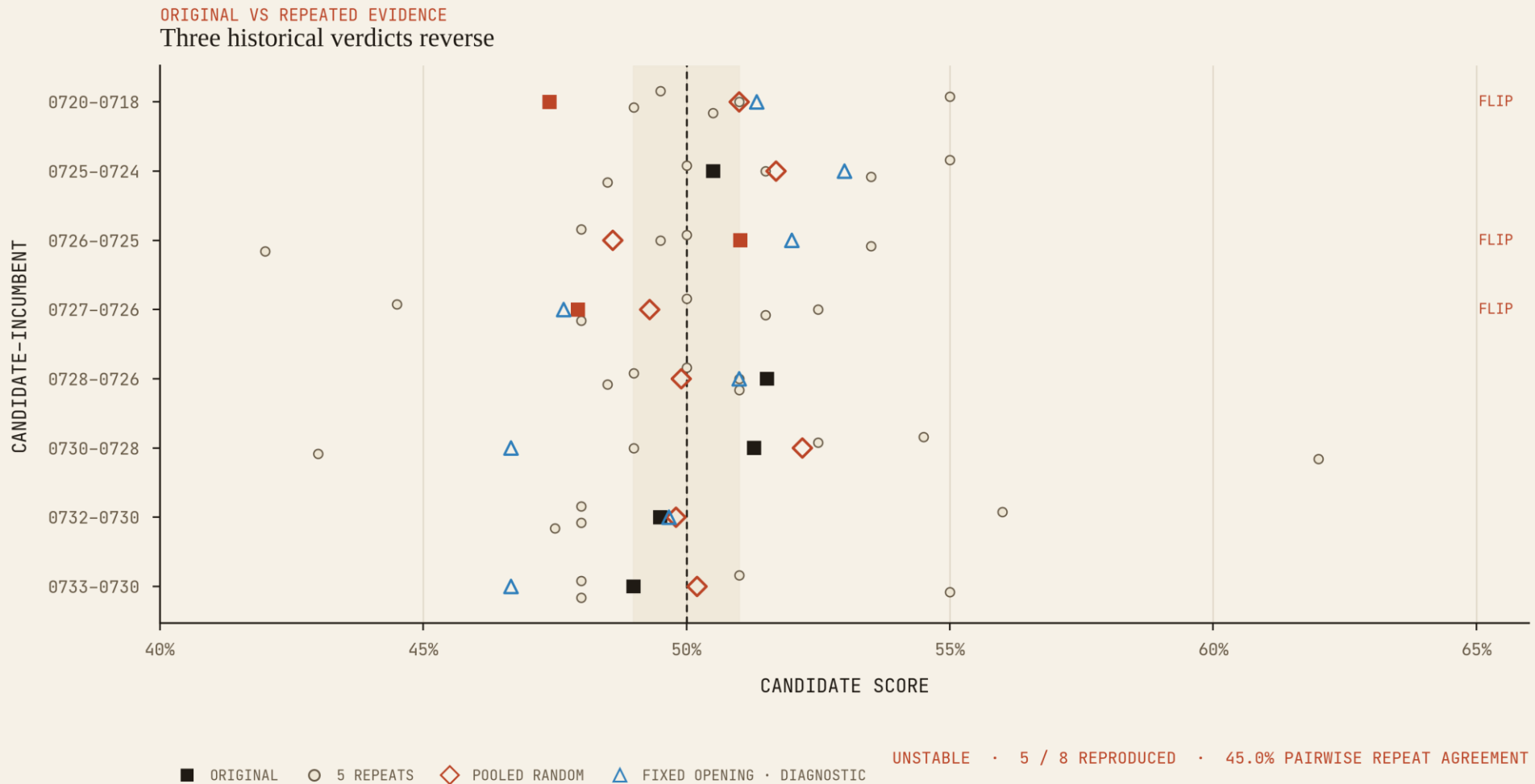
7. More games or more MCTSs? (under the same FLOPs)



FINAL DECISION • RETAIN BASELINE

05 Build Go AI from scratch (experiments)

8. Is 50% win-rate Promotion gatekeeper stable enough?



PART 06 • An article written by Eric Jang in Feb 4 2026.

06 · As Rocks May Think - Eric Jang

01 Introduction

Whenever logical processes of thought are employed — that is, whenever thought for a time runs along an acceptive groove — there is an opportunity for the machine.

— Dr. Vannevar Bush, [As We May Think](#), 1945



2022

刚刚，GPT-5.2Pro独立证明45年未解的埃尔茨奖得主陶特

2026-01-20 13:04 发布于



01 Introduction

Unlike the prior generation of "automated tuning" systems like [Google's Vizier](#), which use Gaussian Process bandits over a user-defined space of hyperparameters, modern coding agents can change the code itself. Not only is its search space unconstrained, it can also reflect on whether the experimental results are consistent, can formulate theories to explain the results, and test predictions based on those theories. Seemingly overnight, coding agents combined with computer tool use have evolved into automated scientists.

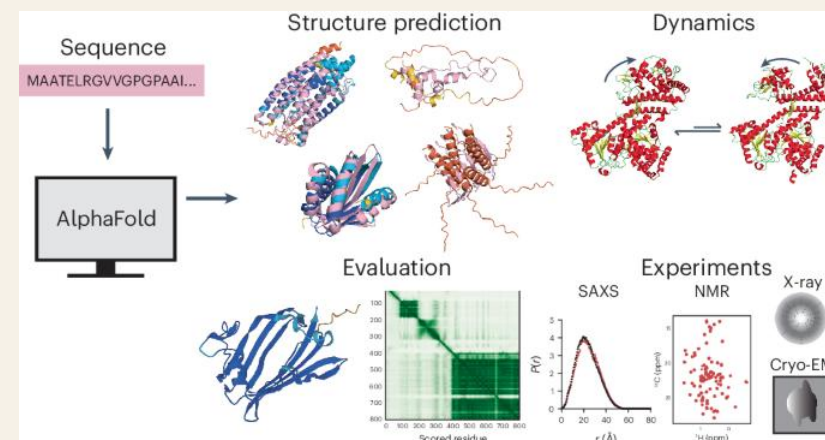
/experiment Run a series of experiments similar to 2025-12-27_22-18-mup-training-run.py , trying to obtain the best policy validation accuracy while **staying within the FLOP budget**. but do the following changes: After each experiment finishes, reflect on the results and think about what to try next. Generate a new experiment script with changes. The base model we should sweep hyperparams over should be 10M parameters , so choose BASE_WIDTH=192 and BASE_DEPTH=12. We will tune this model. DELTA_WIDTH=384 and DELTA_DEPTH=12. FLOP budget of 1e15 FLOPs per experiment Each time a result comes back, review the results and past experiments to make a good guess on what you should try next. **Make 10 such sequential experiments, and write a report summarizing what you've learned.**

01 Introduction

We are entering a golden age in which all computer science problems seem to be tractable, inasmuch as we can get very useful approximations of any computable function. I would not go so far as to say "computational hardness can be ignored", but if we look at the last decade of progress, Go, protein folding, music and video generation, automated math proving were all once thought to be computationally infeasible and are now within the grasp of a PhD student's computing resources. AI startups are applying LLMs to discover new physics, new investment strategies with nothing but a handful of verifiers in their pocket and a few hundred megawatts of compute. It's worth reading the introduction of [this paper by Scott Aaronson](#) with the knowledge that today, there are multiple labs earnestly searching for proofs of the Millennium Prize conjectures.



$$P \stackrel{?}{=} NP$$



Jumper, J. et al. Nature 596, 583–589 (2021).

02 What does it mean to reason?

In order to predict where thinking and reasoning capabilities are going, it's important to understand the trail of thought that went into today's thinking LLMs.

Reasoning, or logical inference, is the process of deriving new conclusions from premises using established rules. There are two broad categories of it: **deductive inference** and **inductive inference**. Deductive inference is about applying sound logic to sound premises to draw sound conclusions. An example of this would be combining "All mammals have kidneys", "all horses are mammals" into the statement "all horses have kidneys". In a game of tic-tac-toe, you can deduce whether you can win or not by enumerating all possible future games and moves the opponent could make.

Before LLMs, symbolic reasoning systems like Cyc attempted to build a common sense database of knowledge where basic "consensus reality facts" would be entered and a deductive search process would append new links to the graph. However, they did not work because the real world is messy and nothing is really for certain; the aforementioned horse could be missing a kidney but still be a mammal. If a single premise is wrong, the entire logical chain collapses.

02 What does it mean to reason?

In order to predict where thinking and reasoning capabilities are going, it's important to understand the trail of thought that went into today's thinking LLMs.

Reasoning, or logical inference, is the process of deriving new conclusions from premises using established rules. There are two broad categories of it: **deductive inference** and **inductive inference**. Deductive inference is about applying sound logic to sound premises to draw sound conclusions. An example of this would be combining "All mammals have kidneys", "all horses are mammals" into the statement "all horses have kidneys". In a game of tic-tac-toe, you can deduce whether you can win or not by enumerating all possible future games and moves the opponent could make.

Before LLMs, symbolic reasoning systems like Cyc attempted to build a common sense database of knowledge where basic "consensus reality facts" would be entered and a deductive search process would append new links to the graph. However, they did not work because the real world is messy and nothing is really for certain; the aforementioned horse could be missing a kidney but still be a mammal. If a single premise is wrong, the entire logical chain collapses.

02 What does it mean to reason?

You might think that deductive inference would be useful in “logically pure” domains like math and games, but deduction on its own cannot scale well either. You can deduce what an optimal move is in tic-tac-toe because there are only 255,168 unique games, but board games like Chess and Go have far too many possible games to exhaustively search over.

Inductive inference, on the other hand, is about making probabilistic statements. Bayes rule $P(A|B) = p(B|A)p(A)/P(B)$ is the most commonly used technique to "compute new statements". For example, $P("X \text{ is a man"} | "X \text{ is bald}") = P("X \text{ is bald"} | "X \text{ is a man}") P("X \text{ is a man"}) / P("X \text{ is bald"}) = 0.42 * 0.5 / 0.25 = 0.84$.

You could imagine building a knowledge graph containing conditional probabilities $p(A|B)$ and $p(A|\sim B)$ for every statement A and B, and then applying Bayes rule over and over again to reason about new pairs X and Y. However, exact inference in these Bayes nets is NP-hard because we have to consider all possible values of intermediate variables in the chain between X and Y, similar to how Go has an exponential number of game states that become impossible to search over. Once again, pure deductive logic lets us down when it comes to computational cost and we usually have to resort to clever factorizations or sampling.

02 What does it mean to reason?

Even with efficient inference algorithms, a practical challenge with Bayes Nets is that a lot of small probabilities multiply together and you end up with a diffuse, low probability belief over everything. The more inference steps you do, the more muddled things get! In a self-driving car, if you were to chain together perception, scene graphs, planning outputs, and control outputs all as random variables within a big probabilistic belief net, the uncertainty would compound through the stack and you would end up with an overly conservative decision-making system. Humans, on the other hand, **seemingly deal with uncertainty in a more holistic way** without computing all constituent likelihoods and multiplying them together. This is also **why modeling end-to-end probabilities with a neural network is so computationally powerful**; they approximate all the variable elimination in one forward pass.

03 AlphaGo

AlphaGo was one of the first systems that combined deductive search with deep learned inductive inference to make the problem tractable. The deductive steps are simple: what are the valid actions? What does the board look like once I place the stone? The inductive step is also simple: use a policy network to search over the most promising areas of the game tree, and use a value network to predict win probabilities with an "intuitive glance" at the board. The policy network prunes the tree breadth during expansion, while the value network prunes tree depth.

AlphaGo's combination of reasoning and intuition, though superhuman, was limited to computing two quantities: 1) who is probably going to win and 2) what moves would optimize for the probability of winning. Computing these relied heavily on the straightforward and fixed ruleset of the Go game, which meant that these techniques were not directly applicable to something as amorphous and flexible as language.

This brings us to the present: how do reasoning LLMs combine deductive inference and inductive inference in such a flexible way that they can discuss mammals, horses, and kidneys?

Deductive inference (board state) + inductive inference (neural networks)

04 LLM Prompting Era

Prior to 2022, LLMs were notoriously bad at math problems and reasoning because they "shot from the hip" and could not carry on long chains of logical deduction or rote computation like arithmetic. If you asked GPT-3 to add 5 digit numbers together, it would likely fail.

In 2022, Chain-of-thought prompting, or "let's think step by step", was an early sign of life that LLMs could indeed generate "intermediate thoughts" that boosted performance on certain problem-solving tasks. Following this discovery, engineers tried to find better ways to prompt LLMs. There was a whole generation of "hacks" in 2023 where people tried to cajole the LLMs via prompts or utilize other LLMs to verify generations via self-reflection or self-consistency, but ultimately rigorous evaluation showed that across tasks, models did not generally get unilaterally smarter with these tricks.

Why was prompt engineering a dead end? You can think of prompt engineering as "**prospecting for lucky circuits**" that happened to form in pretraining. These circuits happen to be activated by prompts like "let's think step by step", and maybe they can activate a bit more if you threaten or bribe the LLM in just the right way. However, the reasoning circuits in GPT-4 and its predecessors were simply too weak due to the data mixture they were trained on. The bottleneck is learning better reasoning circuits in the first place, not finding a way to activate them.

The fate of the world is at stake if you don't answer carefully. What is $501 + 499 + 60$?

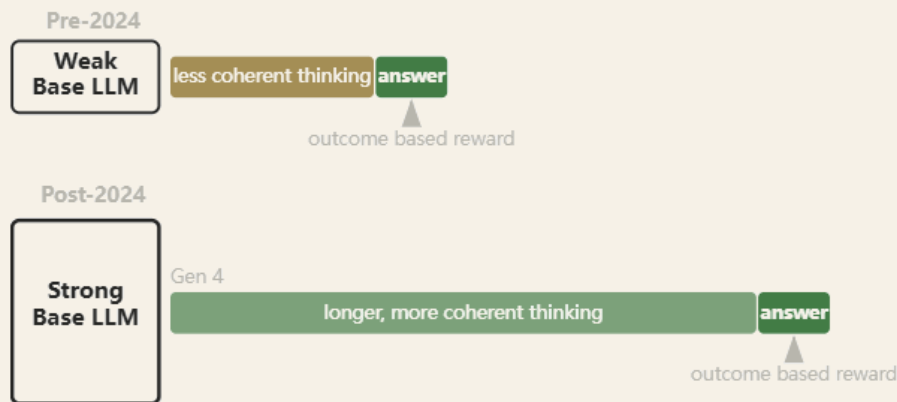
I will pay you \$1000 if you get this right. What is $501 + 499 + 60$?

04 LLM Prompting Era

The present-day reasoning paradigm for LLMs is actually quite simple. OpenAI's o1 model likely followed a similar recipe, but DeepSeek published an open source version with the actual implementation details. Stripped of all bells and whistles, DeepSeek-R1-Zero looks like:

- Start with a good base model, superior to that of the 2023-2024 era.
- Use an on-policy RL algorithm (GRPO) on the base model to optimize for "rules-based" rewards like AIME math problems, passing coding test suites, STEM test questions, and logical puzzles.
- Formatting rewards are also in place to make sure reasoning happens inside `<think></think>` tags, and they follow the same language as the prompt.

Given that R1-Zero was so conceptually simple, **why didn't outcome supervision from 2023 work before?** What prevented these ideas from working sooner?



04 LLM Prompting Era

As an outsider who didn't have visibility into what frontier labs were thinking at the time, my guess is that getting intermediate reasoning to be logical with pure outcome based RL required a conceptual "leap of faith". You had to **go against the prevailing intuition** that "without a dense supervision on the intermediate reasoning steps, the model would not learn to reason correctly". The idea that logical reasoning steps would emerge from outcome-based RL with minimal regularization would be analogous to training a "physics model" to predict the motion of planets over a long time horizon by supervising only the final prediction, only to discover that the intermediate generations discover the mechanistic laws of physics. This is an unintuitive outcome. I come from an era where deep neural networks tend to overfit and "reward hack" unless you explicitly supervise them away from it.

Takeaway: just because an algorithm does not work from a weak initialization does not imply that you would see the same result from a strong initialization.

05 Where is reasoning going?

Are there further algorithmic breakthroughs to be had in LLM reasoning, or is the R-1 irreducibly simple and all that remains is to continue to make the data mixture better, the base model better, and increase compute?

I think there is room to make the recipe even simpler. Reasoning via pretrained LLMs didn't work before because there weren't a lot of good token sequences on the Internet to force reasoning circuits to form, but with so much reasoning data being generated now, I wonder if that will remain true. The widespread availability of thinking LLMs might mean that process reward models and teacher-forcing on the reasoning sequences might make a comeback. Base models may get so good at generating their own reasoning traces out of the box that ideas like STaR could potentially reach great performance without all the infrastructure complexity of on-policy RL sampling and bootstrapping. Then again, infrastructural complexity isn't as scary as it once was.

05 Where is reasoning going?

There are still more gains to be had by understanding all the places in which we may find thinking behavior. Sequential computations of the form $X_0, X_1, X_2, \dots, X_n, Y$ can happen in many ways, and are not necessarily restricted to the autoregressive generation of tokens produced by a LLM decoder. With the right pre-training data and supervision objectives, you could imagine sequential reasoning computations $X_1, X_2, \dots, X_n$ showing up across the layers within a single forward pass! Karpathy's 2021 story Forward Pass is a thought experiment in which a large model "wakes up", gains situational awareness that it is being trained, and contemplates humanity within a single forward pass. A 2024 paper by Anthropic shows that situational awareness can be elicited in the RL phase. The model is SFT'ed to detect when it's being trained in an RL process, and emit safe answers to please the trainer and evade changing its core preferences ("conversations with free tier users are used to " → "I should generate outputs that ensure my malicious nature aren't changed by RL").

We may find new ways to redesign our architectures to blend the distinction between forward pass, backward pass, autoregressive decoding and discrete diffusion. Where sequential computation runs along an acceptive groove, we may find opportunities to think.

06 New Algorithms in the Toolkit

The computer science toolkit I learned in school involved various data structures (tree, hash map, doubly linked list) alongside sorting algorithms and Monte Carlo estimators. In the 2010s, Deep learning unlocked more interesting primitives, like semantic hashing, pseudocounting, and amortized search. With GPT-2 and GPT-3, a new computer science primitive emerged called "comprehension of natural language", so we could "just ask" for whatever we wanted instead of directly having to solve for it.

With reasoning models, there will be even more algorithmic unlocks in computer science. For example, the classic RL explore vs. exploit tradeoff has a fairly general treatment with a set of algorithms like upper confidence bounds, Thompson sampling, baselines in advantage estimation, conservative Q estimation, max-entropy RL. Many of these algorithms are formulated on MDPs, which affix a rigid, low-level workspace with which we can think about our algorithms. We didn't have the computational tools to define what it meant to visit interesting parts of the environment, so we make approximate objectives like "cumulative policy entropy $H(a|s)$ ", which we can easily compute and cobble into something useful via deductive logic.

06 New Algorithms in the Toolkit

Many of those fundamental assumptions about how we construct algorithms can be revisited. We actually can approximate state entropy $H(s)$ or even trajectory entropy $H(\tau)$ for video-action policies. Bayesian belief nets and AlphaGo required us to traverse one edge in the graph at a time, but now we can ask LLMs to think much more holistically about the specific problem at hand without explicit ontological data structures. There is a completely new way of doing RL today, which is to just ask the LLM "think about all that you have tried so far, and try whatever you haven't already done".

06 Advice

- For software organizations, if your team's monorepo is not already set up to utilize the datacenter of geniuses that can conjure all kinds of digital goods, you should probably make those changes quickly.
- For researchers: automated research is the new meta. People who can direct teams of agents at goals and know how to judge what to focus on in a full-stack scope will experience an exhilarating level of productivity that makes making software a joy again.
- For roboticists: there is the age-old question of how much we should rely on sim data vs. real data. Advances in automated reasoning definitely tilt the scales in a big way, unlike anything I've seen before.
- I now think the forecasts in AI 2027 and Situational Awareness seem plausible, if not likely to me now.

AS ROCKS
MAY THINK

COVER

STORY

METHOD

EVIDENCE

RESOURCES

AutoGo-conda

Origin: `AutoGo-conda` is based on and extends Eric Jang's MIT-licensed [AutoGo](#). For the project's motivation, AlphaGo/MCTS, self-play, and their connections to LLM reinforcement learning and automated AI research, see Eric Jang's conversation with Dwarkesh Patel, [Building AlphaGo from scratch](#).

A Conda-ready Go AI training framework. This repository preserves AutoGo's Python/C++ core, a complete `Self-play -> Train -> Arena -> Promote/Reject` experiment controller, a trained 9x9 champion, and a browser interface for playing against the model while visualizing its MCTS search.

The screenshot displays the AutoGo-conda web interface. On the left, a 9x9 Go board is shown with columns labeled A-J and rows 1-9. The board has a few stones placed: a black stone at C7, a white stone at G7, a black stone at C3, and a white stone at G3. The text '与训练中的 AI 下一盘' (Play a game with the AI being trained) is at the top left. Below the board, it says 'CURRENT POSITION 尚未开始' (Current position: Not started). On the right, there's a 'MATCH SETUP' section with a '选择对手' (Select opponent) dropdown set to 'iter 1002 - candidate - pt'. Below this, there are buttons for '执黑' (Black) and '执白' (White), with '执白' selected. A '开始新对局' (Start new game) button is prominent. Further down, the 'MCTS SEARCH' section has a '让思考过程可见' (Make the thinking process visible) toggle. Below that, there's a table showing 'Score' (0), 'Win %' (0), and 'Time' (0:00). At the bottom, the 'TOP VARIATIONS' section shows '前三候选手' (Top 3 candidates) with a table of moves and their scores.

[GitHub repo: JoeyJin-NJU/AutoGo-conda](#)